

Formation FPGA

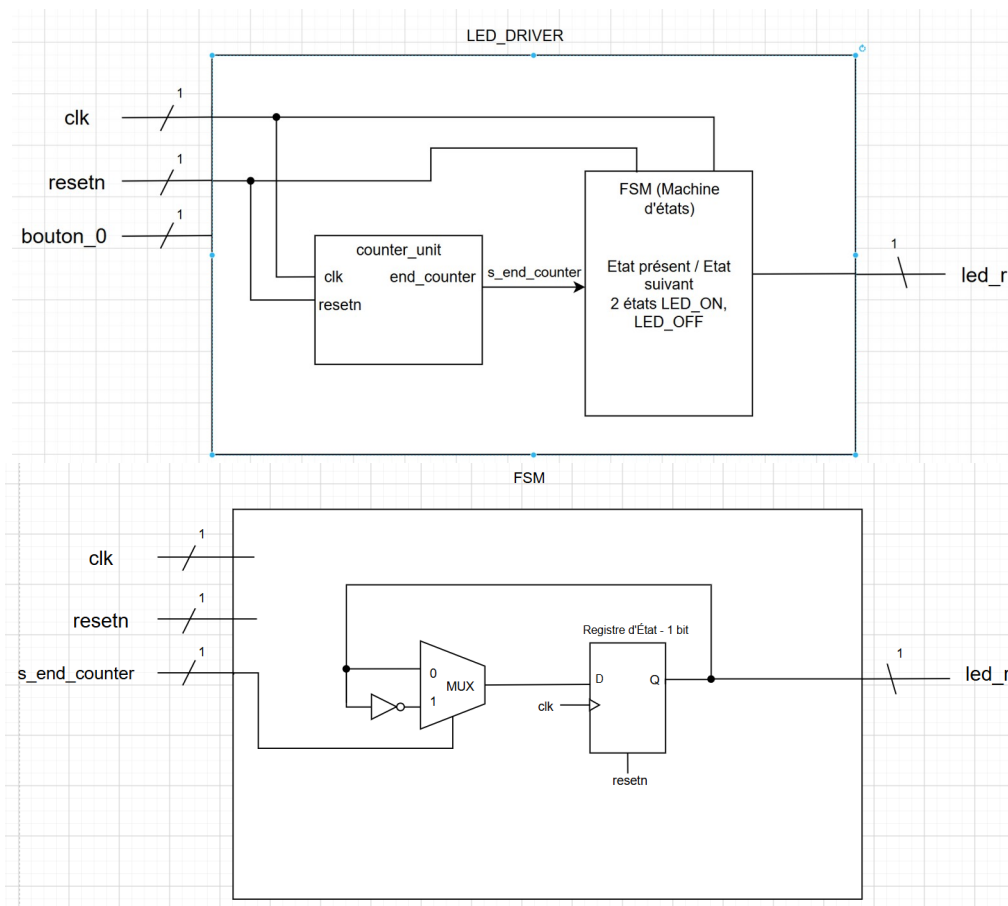
TP4 : LED driver et mémoire

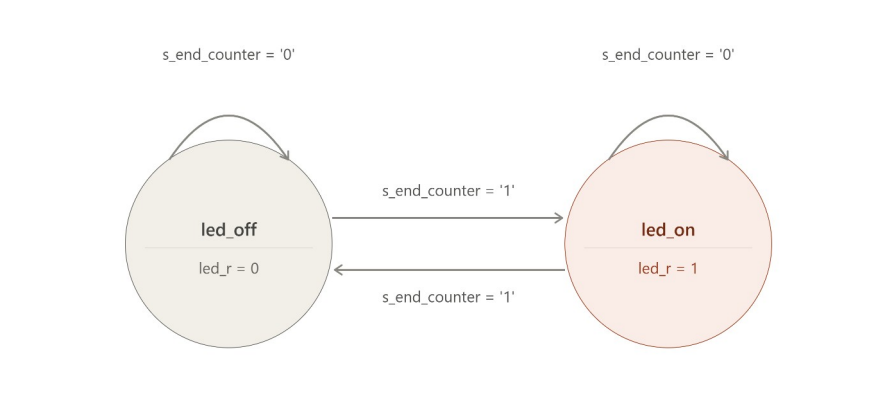
1. Introduction

L'objectif de ce quatrième TP est la modélisation et la réalisation d'une machine d'états sous Vivado via un module de gestion de LEDs, '**LED_Driver**'. La manipulation consiste en sa description matérielle (compteur + unité de contrôle FSM), suivie de sa validation par simulation fonctionnelle (Testbench) et de l'analyse des contraintes temporelles. Enfin, l'implémentation et la programmation du Bitstream sur la carte Cora Z7 permettront la démonstration physique du contrôle du cycle allumé/éteint de la LED.

2. PARTIE I

2.1. Question 1

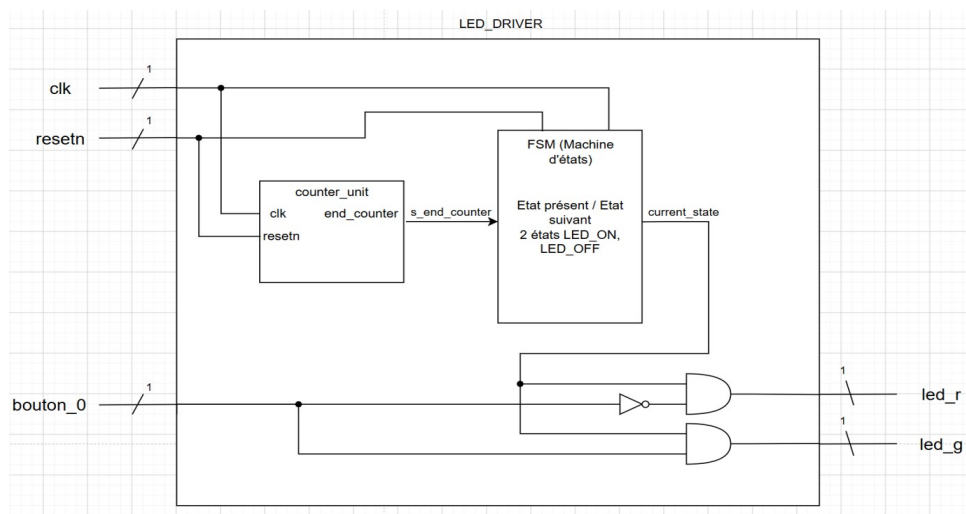




Le schéma RTL ci-dessus représente les premières bases de notre application qui consiste ici à uniquement allumer une LED selon un état donné. Dans un premier temps, une machine sur 2 états est suffisante pour le cycle de clignotement total d'une seule LED. Le module « **counter_unit** » et son signal de sortie '*s_end_counter*' serviront de repère pour la transition entre l'état **LED_OFF** (LED éteinte) et la **LED_ON** (LED allumée). Le registre de la FSM nécessite donc 1 bit qui changera de valeur avec le multiplexeur et le signal de commande correspondant.

Cette machine d'états peut également être réutilisée, en modifiant les sorties en fonction de des entrées ou des signaux internes, selon l'application à réaliser par la suite.

2.2. Question 2



Pour cette deuxième étape, un bouton en entrée de l'entité ainsi qu'une autre broche physique de la LED en sortie seront implémentées. Dans cette configuration, l'état actuel du système est nécessaire afin d'orienter le processus vers le bon affichage de LED.

Une analyse plus approfondie de cette partie révèle que la LED rouge s'allume seulement quand le bouton 0 est au niveau bas (valeur = '**0**'), et l'inverse pour la LED verte. Cela nous amène à cette configuration de tables de vérité :

current_state	bouton_0	LED rouge
'0' (LED_OFF)	'0'	'0'
'0' (LED_OFF)	'1'	'0'
'1' (LED_ON)	'0'	'1'
'1' (LED_ON)	'1'	'0'

current_state	bouton_0	LED verte
'0' (LED_OFF)	'0'	'0'
'0' (LED_OFF)	'1'	'0'
'1' (LED_ON)	'0'	'0'
'1' (LED_ON)	'1'	'1'

Les sorties obtenues permettent ainsi de designer les connexions avec les portes **AND**.

2.3. Question 3

```

entity led_driver is
  generic(
    TOGGLE_DELAY : integer := 100000000 --Horloge à 100 MHz -> 1 seconde pour chaque changement d'état
  );
  port (
    clk      : in std_logic;
    resetn   : in std_logic;
    bouton_0 : in std_logic;
    led_r     : out std_logic;
    led_g     : out std_logic --Broches physiques pour chaque couleur de la LED RGB
  );
architecture behavioral of led_driver is

  component counter_unit is
    generic (
      TOGGLE_DELAY : integer
    );
    port (
      clk      : in std_logic;
      resetn   : in std_logic;
      end_counter : out std_logic
    );
  end component;

  type state is (led_off, led_on); --a modifier avec vos etats

  signal current_state : state; --etat dans lequel on se trouve actuellement
  signal next_state : state; --etat dans lequel on passera au prochain coup d'horloge

  signal s_end_counter : std_logic;

begin

  --Signal "Enable" toujours spécifiée à 1 - Pas de mention pour utilisation spécifique

  -- Instanciation du module cycle_counter
  Counter : counter_unit
    generic map (
      TOGGLE_DELAY => TOGGLE_DELAY
    )
    port map (
      clk      => clk,
      resetn   => resetn,
      end_counter => s_end_counter

  --Unité de controle synchronisant la machine d'états à l'horloge
  Control_Unit : process(clk,resetn)
  begin
    if(resetn = '0') then
      current_state <= led_off;

    elsif(rising_edge(clk)) then
      current_state <= next_state;

    end if;
  end process Control_Unit;

  -- FSM
  State_Machine : process(current_state, s_end_counter)
  begin
    case current_state is
      when led_off =>
        if(s_end_counter = '1') then
          next_state <= led_on;
        else
          next_state <= led_off;
        end if;

      when led_on =>
        if(s_end_counter = '1') then
          next_state <= led_off;
        else
          next_state <= led_on;
        end if;

    end case;
  end process;

```

```

end process State_Machine;

--Affectation des sorties
Outputs : process(current_state, bouton_0)
begin
  case current_state is
    when led_off =>
      led_r <= '0';
      led_g <= '0';
    when led_on =>
      if(bouton_0 = '1') then
        led_g <= '1';
        led_r <= '0';
      else
        led_g <= '1';
        led_r <= '0';
      end if;
    end case;
  end process Outputs;
end process State_Machine;

```

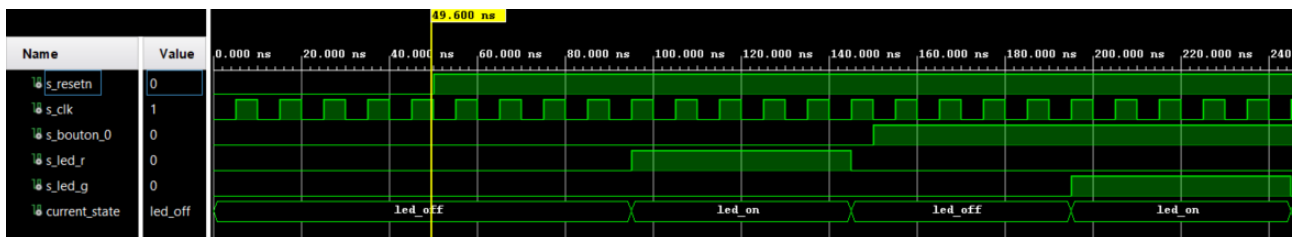
Le code VHDL ci-dessus retranscrit notre schéma RTL théorique permettant dans un premier temps de synchroniser notre machine d'états avec l'horloge dans un premier process « **Unité de contrôle** ».

Un second process « **State_Machine** » pour la logique combinatoire du signal « *next_state* » afin d'orienter le système selon les transitions données, ici la fin du compteur pour passer d'un état à un autre.

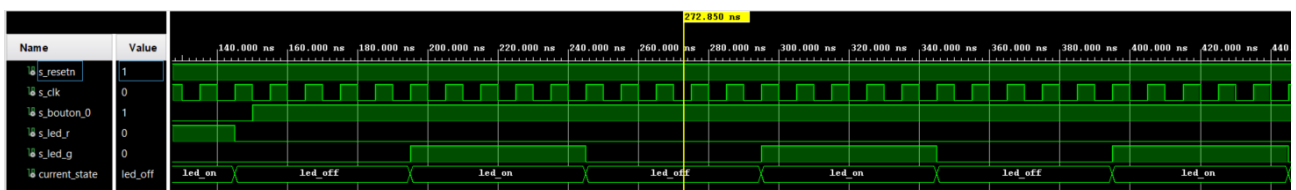
Le dernier process « **Outputs** » correspond donc à l'affectation des sorties LEDs selon la valeur de l'entrée '*bouton_1*' et de l'état actuel du système.

2.4. Question 4

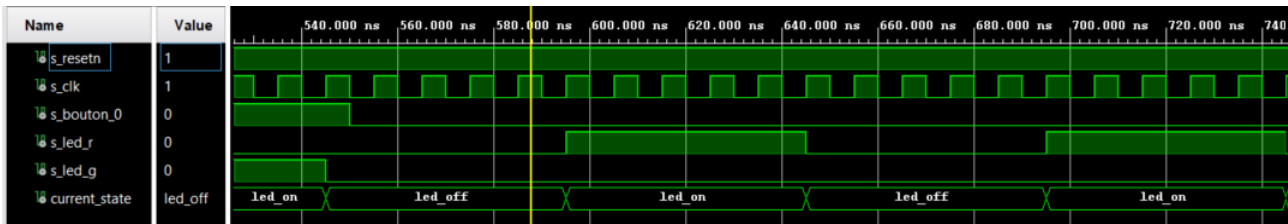
Le testbench de cette étape permet de vérifier la cohérence de notre machine d'états avec les cycles de clignotements de la LED rouge ou de la LED verte quand le bouton est maintenu ou relâché.



La première phase du chronogramme montre la mise à 0 puis à 1 du '*resetn*' afin d'initialiser toutes les variables du circuit. L'application débute avec le clignotement de la LED rouge avec le bouton_0 au niveau bas, c'est à dire à 0.



Avec la mise à l'état haut du bouton_0, la LED verte est utilisée pour les cycles de clignotements, et ceci est valable durant un appui prolongé. Une particularité qui peut être modifiée afin de ne prendre en compte qu'un seul appui peu importe la durée d'impulsion. Ainsi tant que le bouton_0 est à **1**, l'affichage sur carte concernera uniquement la LED verte.



La dernière phase représente un retour au clignotement de la LED rouge, validant ainsi la description VHDL de l'étape actuelle.

Quelques exemples de conditions « assert » ont également été ajoutées afin de relever des erreurs si nécessaire.

```
assert (s_led_g = '0') report "- ERREUR - La LED verte est active au repos" severity error;
report "- INFO - Phase 1 : Attente du clignotement rouge" severity note;
wait for period * 20;
```

Pour la première image par exemple, une erreur est indiquée dans la console si la LED utilisée par défaut n'est pas la rouge.

```
assert (s_led_g = '0') report "- ERREUR - La LED verte s est rallumee apres le relachement !" severity error;
wait for period * 20;
```

Pour la deuxième image par exemple, une erreur est indiquée si la LED verte est toujours utilisée pour l'affichage après la mise à **0** du signal 'bouton_0'.

2.5. Question 5

Afin d'obtenir une seule impulsion quand le bouton est maintenu, il faut partir du principe qu'une mémorisation de l'état précédent du bouton est nécessaire. Cette fonction est possible avec l'aide d'une bascule D qui va ainsi mémoriser avec l'aide d'un autre signal la valeur précédente du bouton qui sera donc comparée avec sa valeur à l'instant présent. En approfondissant au niveau logique, on retrouve cette équation :

$$\text{nouveau_bouton_0} = \text{bouton_0} \text{ AND } (\text{NOT bouton_0_precedent})$$

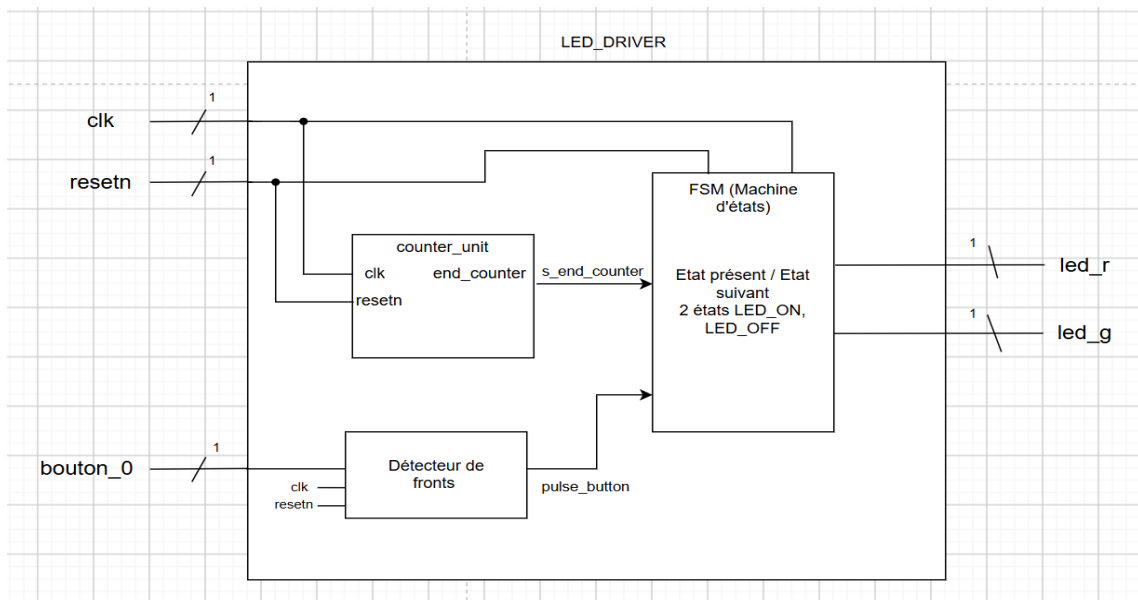
Dans le cas trivial où les deux variables d'entrée sont à **1**, représentant donc un appui prolongé sur le bouton, le nouveau signal reste à **0**.

La seule situation avec un envoi de la valeur '**1**', le registre doit avoir enregistré au préalable la valeur 0, c'est à dire au tout début du circuit lors d'un premier appui, ou bien lors d'un appui suite à un relâchement du bouton. Cette approche justifie donc la logique combinatoire du nouveau signal lié à la mémorisation d'une variable, qui sera appelé « **pulse_button** » dans la description RTL et VHDL.

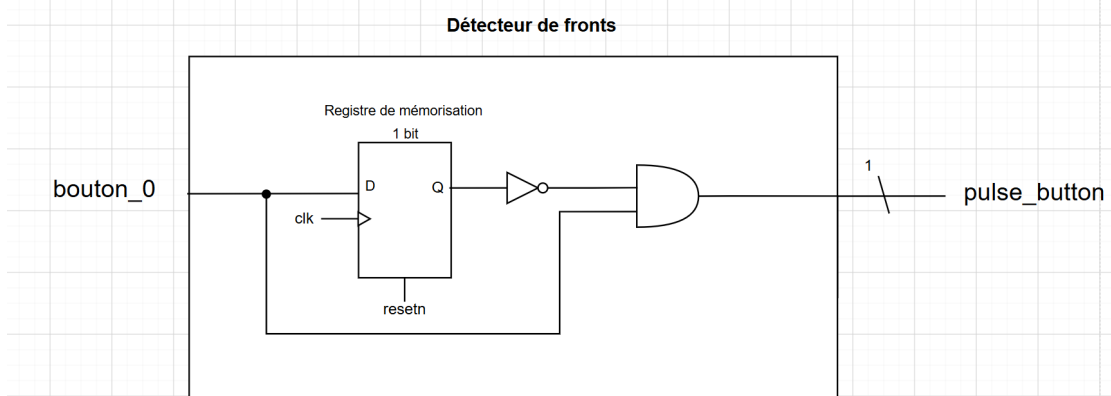
2.6. Question 6

Une machine d'états est toujours constituée de 3 éléments indissociables : la logique d'état futur, les registres d'état, et la logique combinatoire de sortie.

Le schéma RTL global vue du top level, *LED Driver*, représente les trois blocs principaux :



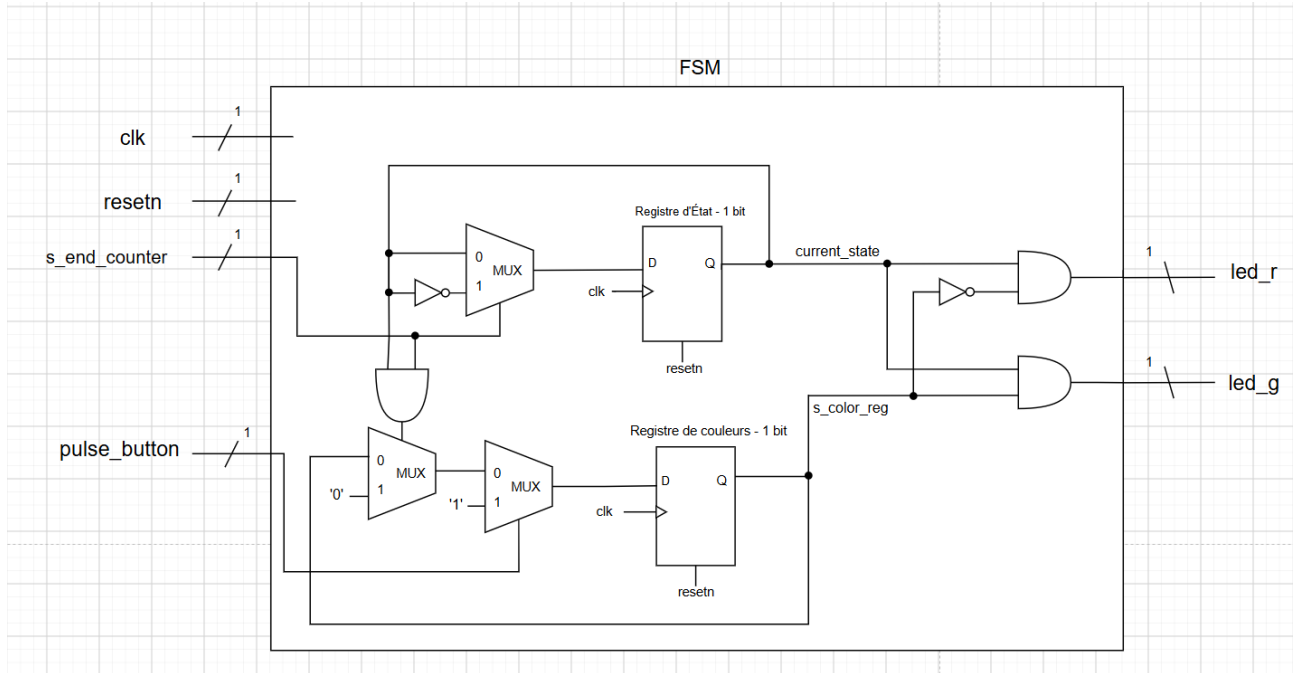
Le mécanisme mis en place pour gérer les impulsions de bouton, *Détecteur de fronts*, prend en compte la logique combinatoire expliquée précédemment. Le bloc *FSM* contient toute la logique d'une unité de contrôle à savoir la synchronisation des états « **LED OFF, LED ON** », les transitions entre ceux-ci, et les sorties pour l'affichage des LEDs.



La représentation ci-dessous prend bien en compte l'équation à l'aide d'un registre de mémorisation:

$$pulse_button = bouton_0 \text{ AND } (\text{NOT } bouton_0_precedent)$$

Le signal 'pulse button' est utilisé comme un signal de commande pour le bloc FSM, représenté en profondeur ci-dessous :



- La première partie permettant de gérer les états actuels et futurs avec le multiplexeur et le registre d'état n'est pas modifiée.
- Un registre de couleurs sur 1 bit est ajoutée afin de simplifier la gestion des LEDs. Concrètement, deux conditions sont nécessaires :
 - Une impulsion du signal 'pulse button' permet de mettre le signal interne 's_color_reg' à **1** pour l'allumage de la LED verte. Étant la première condition à vérifier en priorité, on place le multiplexeur au plus proche de la bascule.
 - Afin d'éviter que le signal 's_color_reg' reste au niveau haut suite à un cycle ON/OFF, il doit être remis à **0** à la fin du cycle. La condition est respectée si l'état actuel est « **LED ON** » et le signal 's_end_counter' est à **1**, signifiant que le cycle est terminé. L'entrée de la bascule prend donc la valeur **0**.
- La troisième partie concerne la gestion des sorties physiques nécessitant les signaux 'current_state' pour l'état actuel ainsi que la valeur du registre de couleurs.

L'affichage est gérée avec la même configuration (*voir tables de vérité .Question 2*) présentée précédemment avec le bouton 0 de la carte, à la seule différence que l'entrée physique 'bouton 0' est remplacée par le signal interne 's_color_reg'.

2.7. Question 7

```
type state is (led_off, led_on); --a modifier avec vos etats

signal current_state : state; --etat dans lequel on se trouve
signal next_state : state; --etat dans lequel on passera
signal pulse_button, s_prev_button_0 : std_logic := '0';
signal s_color_reg : std_logic;
signal s_end_counter : std_logic;

--Unité de controle synchronisant la machine d'états à l'horloge
Control_Unit : process(clk, resetn, pulse_button)
begin
    if(resetn = '0') then

        current_state <= led_off;

    elsif(rising_edge(clk)) then
        s_prev_button_0 <= button_0;
        current_state <= next_state;

        if(pulse_button = '1') then
            s_color_reg <= '1';

            elsif (current_state = led_on and s_end_counter = '1') then
                s_color_reg <= '0';
            end if;
        end if;
    end process Control_Unit;

-- Détecteur de front montant
pulse_button <= button_0 and (not s_prev_button_0); -- Pour éviter le bouton maintenu à '1'

--Affectation des sorties
Outputs : process(current_state, s_color_reg)
begin
    case current_state is

        when led_off =>
            led_r <= '0';
            led_g <= '0';
        when led_on =>
            if(s_color_reg = '1') then
                led_g <= '1';
                led_r <= '0';
            else
                led_g <= '0';
                led_r <= '1';
            end if;
        end case;
    end process Outputs;
```

Le code VHDL implémenté ci-dessus reprend le design du schéma RTL initial mais incluant également le détecteur de fronts pour la gestion du maintien du bouton physique avec la logique combinatoire liée au signal 'pulse_button', placé en dehors d'un quelconque process synchrone et permettant une meilleure réactivité lors d'un changement de valeur. Les conditions du process « **Control Unit** » sont conformes aux nouveaux éléments ajoutés dans le schéma RTL.

Le testbench et la simulation par chronogrammes vont permettre de visualiser cette gestion des impulsions du bouton et la gestion correcte des LEDs.

```

-- 1) Initialisation sous Reset
s_resetrn <= '0';
s_button_0 <= '0';
wait for period * 5;

s_resetrn <= '1';
wait for period * 10;

-- On verifie que la LED verte n'est pas allumee par erreur au debut
assert (s_led_g = '0') report "- ERREUR - La LED verte est active au repos" severity error;
report "- INFO - Phase 1 : Attente du clignotement rouge" severity note;
wait for period * 20;

-- Test : Appui prolonge sur le bouton
s_button_0 <= '1';

-- Premier cycle de clignotement : la LED verte doit s'allumer/clignoter
wait for period * 10;

-- La LED Rouge ne doit pas bloquer le systeme
assert (s_led_r = '0') report "- ERREUR - La LED rouge est restee active juste apres 1 appui" severity error;

-- Attente d'un long moment (bouton toujours maintenu a '1')
wait for period * 30;

-- Verification du blocage : la LED verte doit s'etre eteinte et la rouge doit avoir repris
assert (s_led_g = '0') report "- ERREUR - La LED verte continue de clignoter malgre le filtre" severity error;

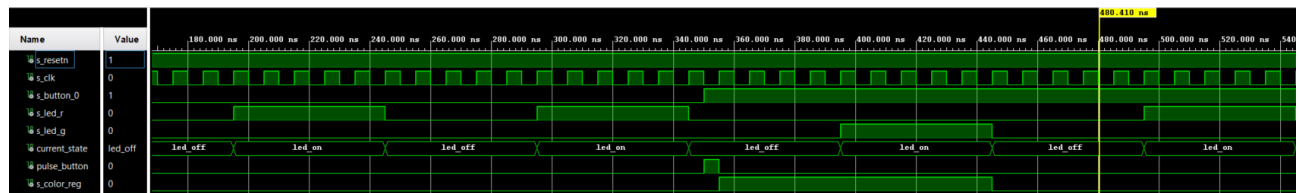
-- 4) Relachement du bouton
s_button_0 <= '0';
wait for period * 10;

-- Verification que la FSM reste bien sur la LED rouge (la verte doit rester a 0)
assert (s_led_g = '0') report "- ERREUR - La LED verte s est rallumee apres le relachement" severity error;
wait for period * 20;

report "- SIMULATION TERMINEE -" severity note;
wait;

```

L'objectif est donc de vérifier que l'impulsion pour éviter le maintien du bouton est sur une durée courte. La FSM doit également présente un fonctionnement adapté selon la couleur de LED affiché.



Le signal 'pulse_button' passe à 1 lors de l'appui sur le bouton 0, puis repasse aussitôt à 0 au prochain front montant. Cela s'explique par le fait que le signal 's_prev_button_0' prenne la valeur de l'entrée de manière synchrone, et avec la logique combinatoire associée :

$$pulse_button = bouton_0 \text{ AND } (\text{NOT } prev_button_0)$$

$$pulse_button = \underline{1} \text{ AND } (\text{NOT } \underline{1}) = \underline{0}$$

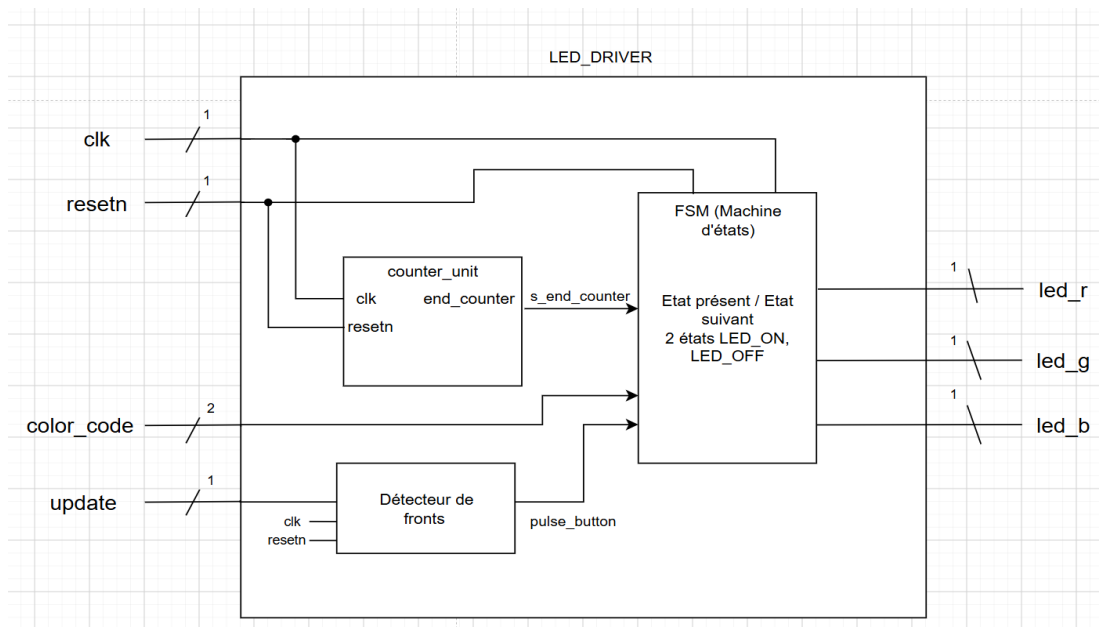
Cette légère impulsion est le flag idéal afin de débiter un cycle pour la LED verte en mettant à 1 le signal 's_color_reg' puis le remettre à 0 à la fin du cycle avec la condition implémentée dans le process de l'unité de contrôle. La simulation est donc validée.

2.8. Questions 8 & 9

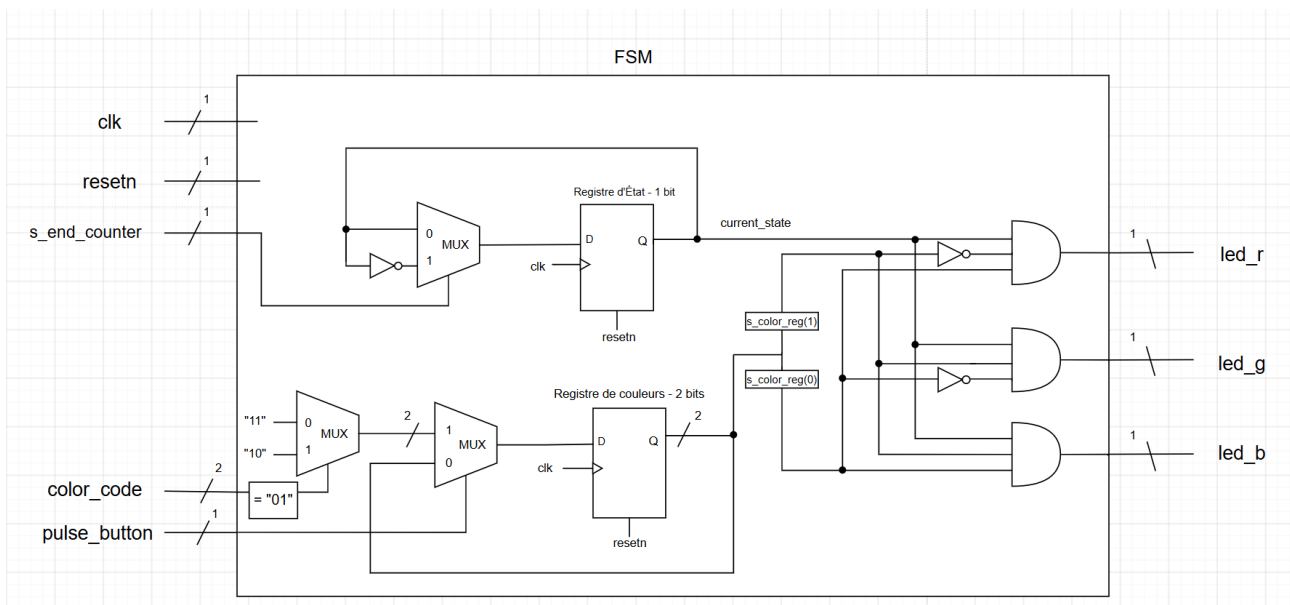
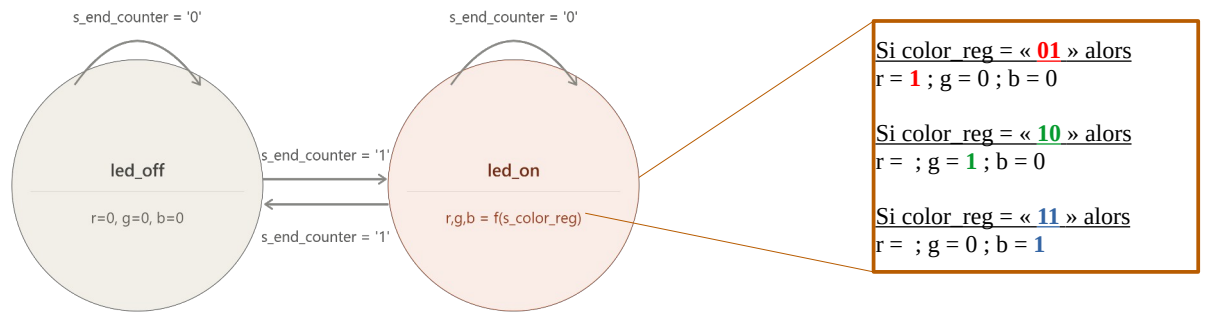
Pour la suite de notre application, l'objectif est d'améliorer la configuration actuelle pour piloter une LED RGB avec l'ajout de nouvelles spécifications. Concrètement, des changements mineurs seront effectués :

- Les sorties physiques seront au nombre de trois (ajout de la LED bleue). Par conséquent, le registre de couleurs sera sur 2 bits et sera donc associé à l'entrée physique 'color_code' (sur 2 bits également) indiquant la couleur correspondante.
- Le signal interne 's_color_code', pour la gestion des sorties LEDs, sera uniquement dépendant de l'entrée 'update' ainsi que de la valeur de l'entrée 'color_code', contrairement à la version précédente avec la condition de fin de cycle ON / OFF et la mise automatique à zéro. Ainsi, le registre de couleurs conserve sa valeur actuelle tant qu'il n'y a pas de nouvelles impulsions avec 'update'.
- Le bouton poussoir peut uniquement jouer sur deux valeurs, **0** et **1**. Par conséquent, l'application fonctionnera uniquement avec les LEDs verte et bleue. La LED rouge pourrait être utilisée comme une sortie par défaut avant une quelconque impulsion sur le bouton 'update' au début de l'application, ou lors d'un reset hardware remettant le système à son état initial.

Ces spécifications amènent à quelques modifications du schéma RTL précédent :



La machine d'états est légèrement modifiée en ajoutant les sorties 3 dédiées :



Le bloc « **FSM** » est la seule partie modifiée du schéma RTL.

Le signal '**color_code**' est implémentée sur deux bits mais avec seulement deux valeurs utilisées « **00** » ou « **01** », un transcodage est nécessaire afin de respecter la table de codes couleur. Si le *bouton 1* est pressé, la valeur envoyée est « **10** » pour la LED verte, sinon « **11** » pour la LED bleue.

Concernant la gestion des LEDs, 3 portes **AND** à 3 entrées sont utilisés avec la logique suivante :

- Si le système est à l'état LED_OFF, les 3 LEDs sont éteintes.
- Si le système est à l'état LED_ON, les sorties respectent la logique suivante selon le code couleur associé :
 - $led_r = current_state \text{ AND } (\text{NOT } s_color_reg(1)) \text{ AND } s_color_reg(0)$
 - $led_g = current_state \text{ AND } s_color_reg(1) \text{ AND } (\text{NOT } s_color_reg(0))$
 - $led_b = current_state \text{ AND } s_color_reg(1) \text{ AND } s_color_reg(0)$

2.9. Question 10

```

entity led_driver is
    generic(
        TOGGLE_DELAY : integer := 1000000000    --Horl.
    );
    port (
        clk          : in std_logic;
        resetn       : in std_logic;
        update       : in std_logic;
        color_code   : in std_logic_vector(1 downto 0);
        led_r        : out std_logic;
        led_b        : out std_logic;
        led_g        : out std_logic --Broches physiq.
    );
end led_driver;

--Unité de controle synchronisant la machine d'états à l'horloge
Control_Unit : process(clk,resetn)
begin
    if(resetn = '0') then

        current_state <= led_off;
        s_prev_update <= '0';
        s_color_reg <= "01"; -- LED Rouge par défaut

    elsif(rising_edge(clk)) then
        s_prev_update <= update;
        current_state <= next_state;

        if(pulse_button = '1') then
            if(color_code = "01") then --Transcodage de valeurs
                s_color_reg <= "10";
            else
                s_color_reg <= "11";
            end if;
        end if;
    end if;
end process Control_Unit;

--Affectation des sorties
Outputs : process(current_state, s_color_reg)
begin
    -- Les LEDs sont éteintes par défaut (1 seule fois au début du process)
    led_r <= '0';
    led_g <= '0';
    led_b <= '0';

    -- Gestion des LEDs
    case current_state is
        when led_off =>
            null; -- Tout reste éteint

        when led_on =>
            if (s_color_reg = "10") then
                led_g <= '1'; -- On passe juste la LED Verte à 1
            elsif (s_color_reg = "11") then
                led_b <= '1'; -- On passe juste la LED Bleue à 1
            else
                led_r <= '1'; -- On passe juste la LED Rouge à 1
            end if;
        end case;
    end process Outputs;
end behavioral;

```

Le code VHDL ci-dessus retranscrit les nouvelles spécifications ajoutées à notre application et les quelques modifications qui vont avec.

Le process de l'affectation des sorties est optimisée afin d'éviter de quelconques latch en définissant une valeur appliquée une fois au début du process. L'intérêt est d'éviter les valeurs indéfinies et de modifier uniquement la valeur de la sortie correspondante, sinon elle reste à son niveau actuel.

Piste d'amélioration : Actuellement, le décodage des LEDs est réalisé de manière totalement combinatoire. Pour cette application, ce choix est optimal car l'œil humain n'est pas capable de visualiser les glitches transitoires de quelques nanosecondes.

Toutefois, dans le cas où ces signaux piloteraient un périphérique rapide comme les bus de données ou bien un protocole de communication par exemple, une amélioration consisterait à insérer une bascule D juste avant chaque broche de sortie. Cela permettrait de :

- Filtrer 100 % des glitches combinatoires.
- Optimiser le temps de propagation entre le front actif de l'horloge (par exemple le front montant) et le moment où le signal sur la sortie devient stable et valide.

2.10. Question 11

```
-- 1) Initialisation par défaut
s_resetrn    <= '0';
s_update     <= '0';
s_color_code <= "00";
wait for period * 4;

s_resetrn    <= '1';

-- Attente automatique du premier allumage physique d'une LED rouge post-reset
wait until (s_led_r = '1' or s_led_g = '1' or s_led_b = '1');

assert (s_led_r = '1' and s_led_g = '0' and s_led_b = '0')
    report "- ERREUR Phase 1 - LED Rouge non active par default"
    severity error;

report "- INFO - : LED Rouge active par default" severity note;
wait for period * 8; -- Avance d'un cycle complet pour rester synchrone

-- 2) Passage à la LED Verte (color_code = "01") + Test d'appui prolonge
wait until rising_edge(s_clk);
s_color_code <= "01"; -- Vert
s_update     <= '1'; -- Front montant sur update

-- Attente que le vert s'allume physiquement
wait until (s_led_r = '1' or s_led_g = '1' or s_led_b = '1');

assert (s_led_r = '0' and s_led_g = '1' and s_led_b = '0')
    report "- ERREUR - Echec passage a la LED Verte"
    severity error;

report "- INFO - OK : Changement vers LED Verte reussi" severity note;

-- Test de l'anti-maintien : le bouton reste appuyé longtemps
s_color_code <= "00"; -- Modification du code couleur pendant que update reste a 1
wait for period * 32;
```

```

-- Verification : Pendant le clignotement vert,
-- Les LEDs Bleue et Rouge ne doivent jamais passer a '1' (que la LED verte soit ON ou OFF)
assert (s_led_b = '0' and s_led_r = '0')
    report "- ERREUR - Instabilite lors de l'appui prolonge (changement de couleur non desire)"
    severity error;

s_update <= '0'; -- Relachement du bouton
wait for period * 12;
report "- INFO - OK : Filtre anti-maintien valide" severity note;

-- 3) Passage à la LED Bleue (color_code = "00") via une impulsion courte
wait until rising_edge(s_clk);
s_color_code <= "00"; -- Bleu
s_update <= '1';
wait for period * 2;
s_update <= '0'; -- Relachement immediat (impulsion)

wait for period * 4;
s_color_code <= "01"; -- Modification du code couleur pour tester la memoire du registre

-- Attente de l'allumage physique
wait until (s_led_r = '1' or s_led_g = '1' or s_led_b = '1');

assert (s_led_r = '0' and s_led_g = '0' and s_led_b = '1')
    report "- ERREUR - Echec passage a la LED Bleue"
    severity error;

report "- INFO - OK : Changement vers LED Bleue reussi" severity note;
wait for period * 20;

-- 4) Fin de simulation
report "- SIMULATION TERMINEE AVEC SUCCES -" severity note;
wait;

```

Le testbench est structuré en 3 parties :

- L'initialisation par le resetn et la vérification de l'allumage de la LED Rouge par défaut.
- Le passage à la LED Verte avec les signaux correspondants ('*update*' et '*color_code*') + Vérification du bon fonctionnement du détecteur de fronts
- Le passage à la LED Bleue et fin de la simulation

De nouvelles instructions de code ont été utilisées afin d'optimiser la simulation :

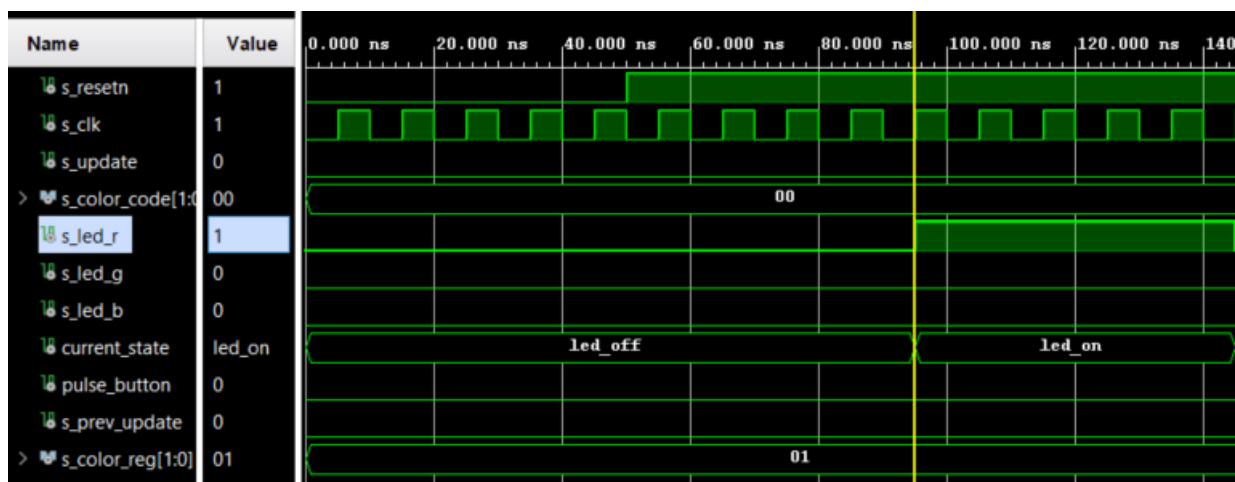
- Utilisation de « *wait until rising_edge(clk)* » avant assignation
 - Placé avant la modification d'un signal, il force l'assignation à s'exécuter juste après le front d'horloge, pendant la phase basse du signal
 - L'entrée a le temps de se stabiliser avant l'arrivée du front d'horloge suivant (Respect du setup time)
 - La FSM voit le signal changer au cycle N et met son registre à jour au cycle N+1, ce qui déclenche l'impulsion à tous les coups
- Utilisation de « *wait until* » couplé aux « *assert* »
 - Le testbench se met en pause et attend la réaction physique du composant (ex: passage d'une LED à '1') au lieu de compter des nanosecondes

- Cela permet une insensibilité aux changements de fréquence, les tests ne cassent pas si la période d'horloge ou la valeur des compteurs est modifiée
- L'instruction *assert* s'exécute instantanément lorsque l'état souhaité apparaît sur les sorties.

2.11. Question 12

Le simulation nous permet de visualiser le bon fonctionnement de l'application en visualisant la gestion des 3 LEDs et des impulsions sur bouton poussoir.

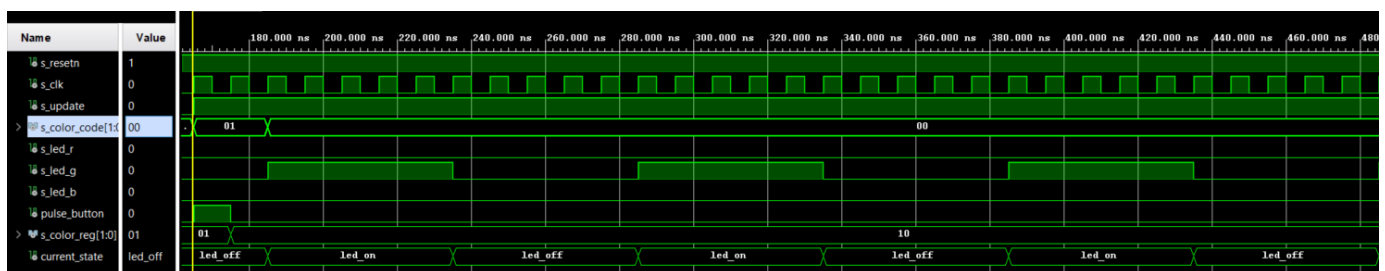
Premier cas : Allumage de la LED rouge (Début de l'application – État par défaut)



Le *reset hardware* est utilisé dans un premier temps pour initialiser le système et démarrer l'application. Comme expliqué précédemment, la LED Rouge est utilisée par défaut avant une quelconque impulsion sur le bouton '*update*' entraînant le changement de couleurs. Les signaux '*color code*' et '*update*' sont à 0 pour ce cas de figure.

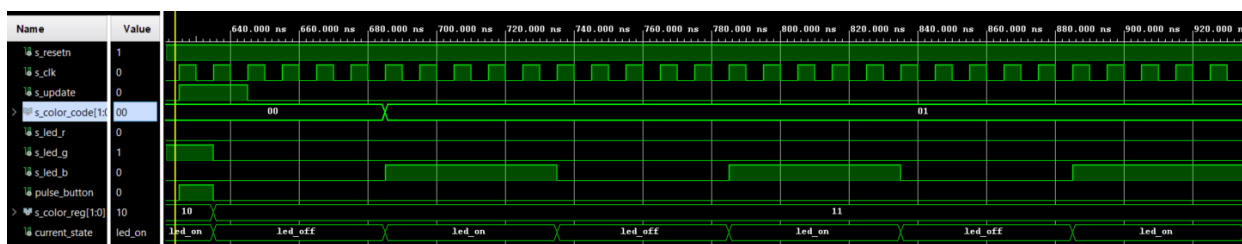
Deuxième cas : Allumage de la LED verte

(Appui sur '*update*' & '*color code*' – Gestion du maintien du bouton)



Pour ce cas de figure, la LED verte est utilisée durant les cycles ON/OFF. Dans un premier temps, le signal 'color_code' change de valeur et passe à « **01** » (signifiant que le bouton est pressé) puis afin d'autoriser cette modification de couleurs, l'enchaînement continue avec la mise à **1** du signal 'update'. Par conséquent, le registre de couleurs prend la valeur « **10** », correspondant au code couleur **VERT**, affectant à la sortie 'led_g' la valeur correspondante. Le détecteur de fronts fonctionne correctement avec l'impulsion brève du signal 'pulse_button' sur une courte durée. Maintenir le signal 'update' à **1** permet de justifier son efficacité : Tant qu'il n'y a pas de nouvelle impulsion sur le signal, le système continue le déroulement des cycles avec la LED verte. Le signal 'color_code' change de nouveau de valeur mais avec l'absence d'une nouvelle impulsion du signal 'update', aucun changement est validé au niveau des LED.

Troisième cas : Allumage de la LED Bleue (Appui sur 'update' uniquement)



Ce cas de figure est similaire au précédent, une mise à l'état haut du signal 'update' et une inaction sur le signal 'color_code' modifie la valeur enregistrée dans le registre de couleurs au front montant suivant. La LED Bleue, correspondant au code couleur « **11** », est activée pour les prochains cycles. La simulation est donc globalement validée.

2.12. Question 13

Avant la synthèse, l'analyse théorique du module prévoit une empreinte matérielle extrêmement faible :

- Logique de contrôle central : 4 bascules D (1 pour s_prev_update, 2 pour s_color_reg, 1 pour la FSM) et environ 5 LUTs pour le transcodage et le décodeur RGB.
- Compteur de temporisation : 28 bits (nécessaires pour le clignotement à 100 MHz).
- Occupation globale théorique : Environ 30 bascules (FFs) et 32 LUTs.
- Interface d'E/S et blocs dédiés : 8 broches d'E/S (5 entrées : clk, resetn, update, color_code[1:0] et 3 sorties LEDs).

Ref Name	Used	Functional Category
LUT5	32	LUT
FDCE	31	Flop & Latch
CARRY4	7	CarryLogic
LUT4	5	LUT
IBUF	5	IO
OBUF	3	IO
LUT3	3	LUT
LUT2	1	LUT
LUT1	1	LUT
FDPE	1	Flop & Latch
BUFG	1	Clock

Les résultats extraits du rapport de synthèse Vivado concordent parfaitement avec nos estimations théoriques :

- Registres (32 FF) : On retrouve exactement les 32 bascules prévues (31 FDCE + 1 FDPE). Les 26 bits du compteur additionnés aux 4 bascules du cœur de contrôle et aux registres de synchronisation atteignent le total estimé
- Logique combinatoire (42 LUT) : Le léger écart par rapport aux 32 LUTs théoriques s'explique par l'architecture interne : Vivado utilise 32 LUT5 pour le compteur et les masques, combinées à 10 LUT plus petites (LUT1 à LUT4) pour la FSM, la détection de front et la logique de sortie
- Chaîne de retenue (7 CARRY4) : L'outil a instancié 7 primitives CARRY4 dédiées pour accélérer l'incréméntation du compteur 28 bits ($7 \times 4 = 28$ bits de capacité) sans consommer de LUT supplémentaires
- Interface d'E/S (8 IO) : Parfaite adéquation avec 5 IBUF (entrées) et 3 OBUF (sorties des LEDs)
- Buffer d'horloge (1 BUFG) : Insertion automatique du buffer global pour distribuer le signal clk sur tout le FPGA sans décalage, ou bien appelé skew

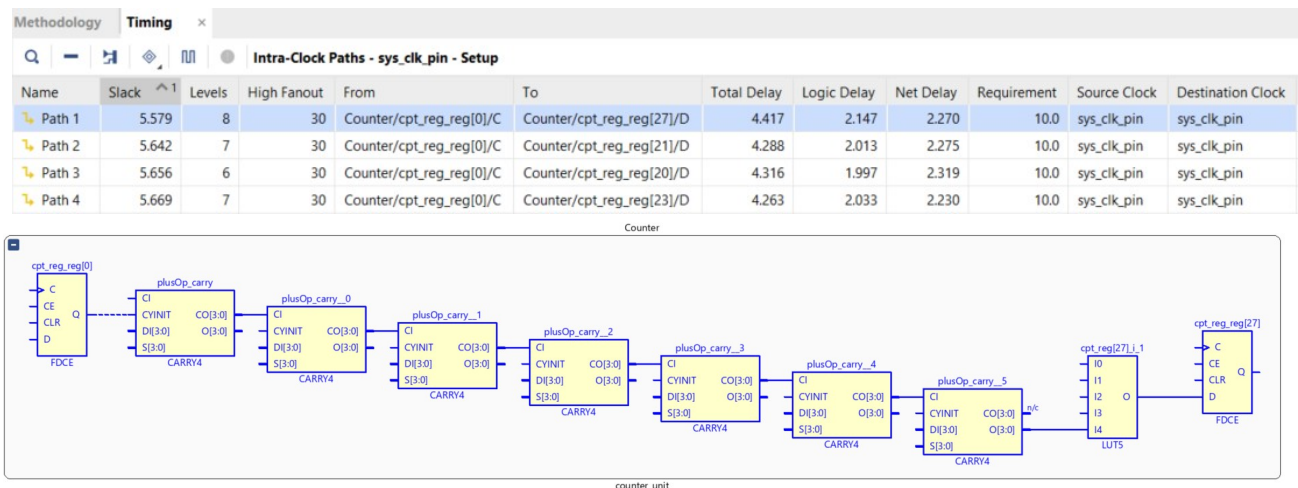
2.13. Question 14

Le rapport d'implémentation final montre un respect parfait des contraintes de timing :

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 5,579 ns	Worst Hold Slack (WHS): 0,196 ns	Worst Pulse Width Slack (WPWS): 4,500 ns
Total Negative Slack (TNS): 0,000 ns	Total Hold Slack (THS): 0,000 ns	Total Pulse Width Negative Slack (TPWS): 0,000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 31	Total Number of Endpoints: 31	Total Number of Endpoints: 33

- Worst Negative Slack (WNS) : Positif (pas de violation de Setup).
- Worst Hold Slack (WHS) : Positif (pas de violation de Hold).



L'analyse temporelle réalisée par Vivado montre que le Path 1 constitue le chemin critique de notre design, présentant à la fois le Slack le plus faible (**5.579 ns**) et le délai de propagation total le plus élevé (**4.417 ns**).

Néanmoins, avec un Slack largement positif de **+5.58 ns** sur une période d'horloge requise de 10.0 ns (horloge système à 100 MHz), l'ensemble de nos contraintes de temps d'établissement (*setup time*) sont parfaitement respectées.

Le chemin critique extrait (visible sur le schéma technologique ci-dessous) s'étend de la bascule de départ `cpt_reg_reg[0]` à la bascule d'arrivée `cpt_reg_reg[27]` au sein de notre module `counter_unit`.

Physiquement, ce chemin correspond à la chaîne d'incrément et de comparaison de notre compteur de 28 bits. Pour propager la retenue du bit de poids le plus faible (**0**) jusqu'au bit de poids fort (**27**), le signal traverse 8 niveaux de logique combinatoire (composés d'une chaîne de blocs de retenue rapide CARRY4 suivie d'une porte LUT5) avant d'atteindre le registre de destination.

2.14. Question 15

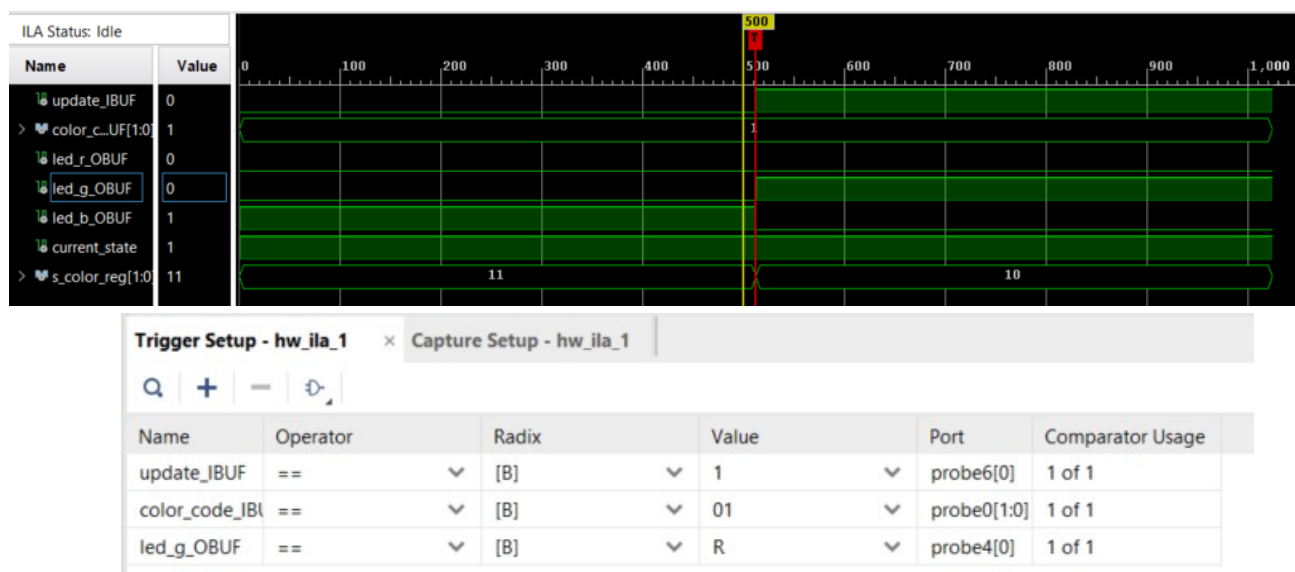
```
## Pmod Header JA
set_property -dict { PACKAGE_PIN Y18      IOSTANDARD LVCMOS33 PULLUP TRUE } [get_ports { resetn }]; #IO_L17E
set_property -dict { PACKAGE_PIN Y19      IOSTANDARD LVCMOS33 PULLDOWN TRUE } [get_ports { color_code[1] }];
```

- **PULLUP** active une résistance interne au FPGA qui tire le signal '*resetn*' au 3.3V ('1'), évitant au circuit d'être bloqué en reset.
- **PULLDOWN** active une résistance interne qui tire '*color_code[1]*' à la masse ('0').

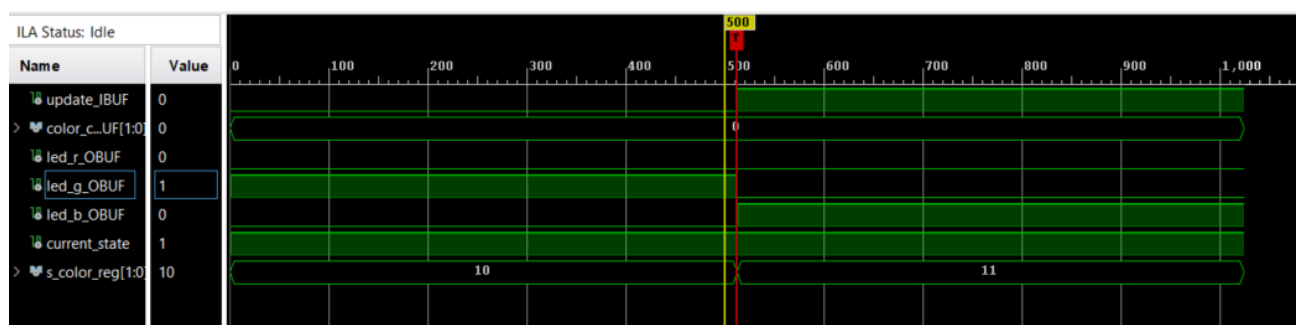
Une démonstration vidéo est également disponible pour montrer le bon fonctionnement de cette partie de ce TP. (disponible dans le fichier README du dossier TP4 qui dirige vers un lien Google drive).

Plusieurs cas de figure ont été testés avec l'utilisation de sonde ILA et à l'aide de différents triggers liés aux entrées/sorties ou signaux internes :

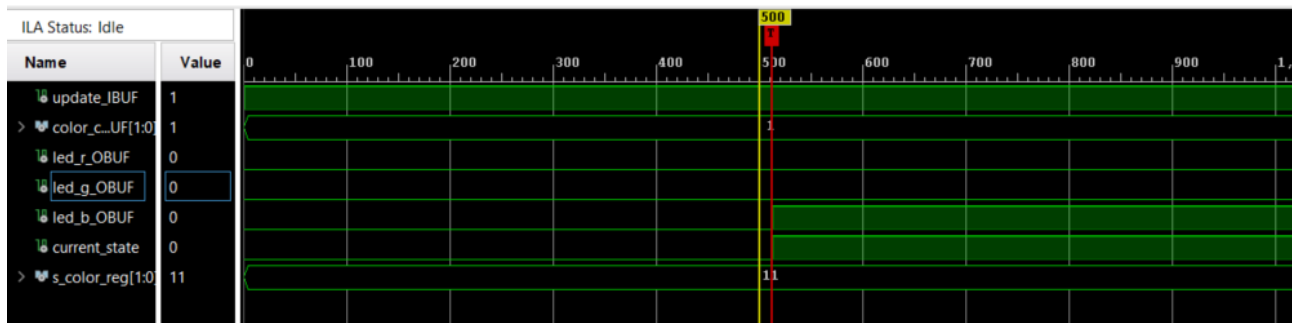
Allumage de la LED Verte → Impulsion update & color code[0] → color_reg = « 10 »



Allumage de la LED Bleue → Impulsion update et non color code[0] → color_reg = « 11 »



Test anti-maintien → Maintien update et impulsion sur color_code[0] → color_reg reste à « 11 » (Bleu) et non « 10 » (Vert)

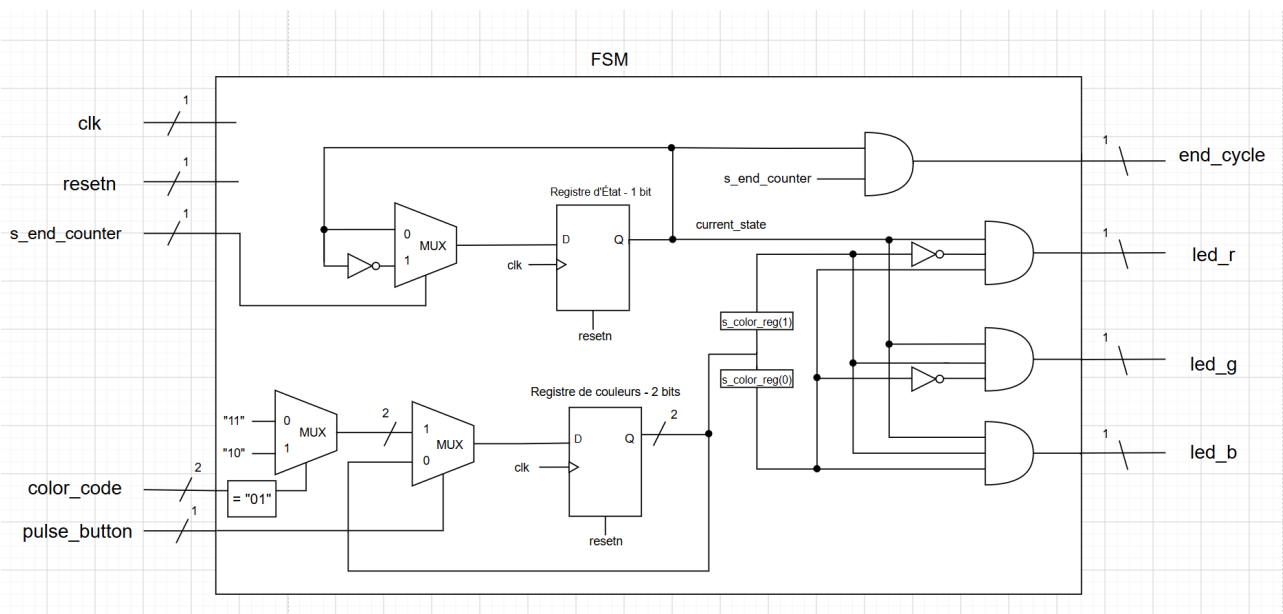


3. PARTIE II

L'objectif de cette seconde partie est de réaliser un design permettant de faire clignoter une LED RGB avec une séquence de couleurs entrées dans une zone tampon à l'aide des boutons. Dans cette partie, le module **LED_driver** de la partie 1 sera associé avec un composant mémoire : la **FIFO**.

3.1. Question 1

Le signal 'end_cycle' est une sortie du module **LED_driver** dont l'objectif sera de donner un top départ pour la lecture et l'envoi de données du **FIFO** vers ce module. Des explications approfondies sont donnés au fur et à mesure sur le fonctionnement global des différentes parties.



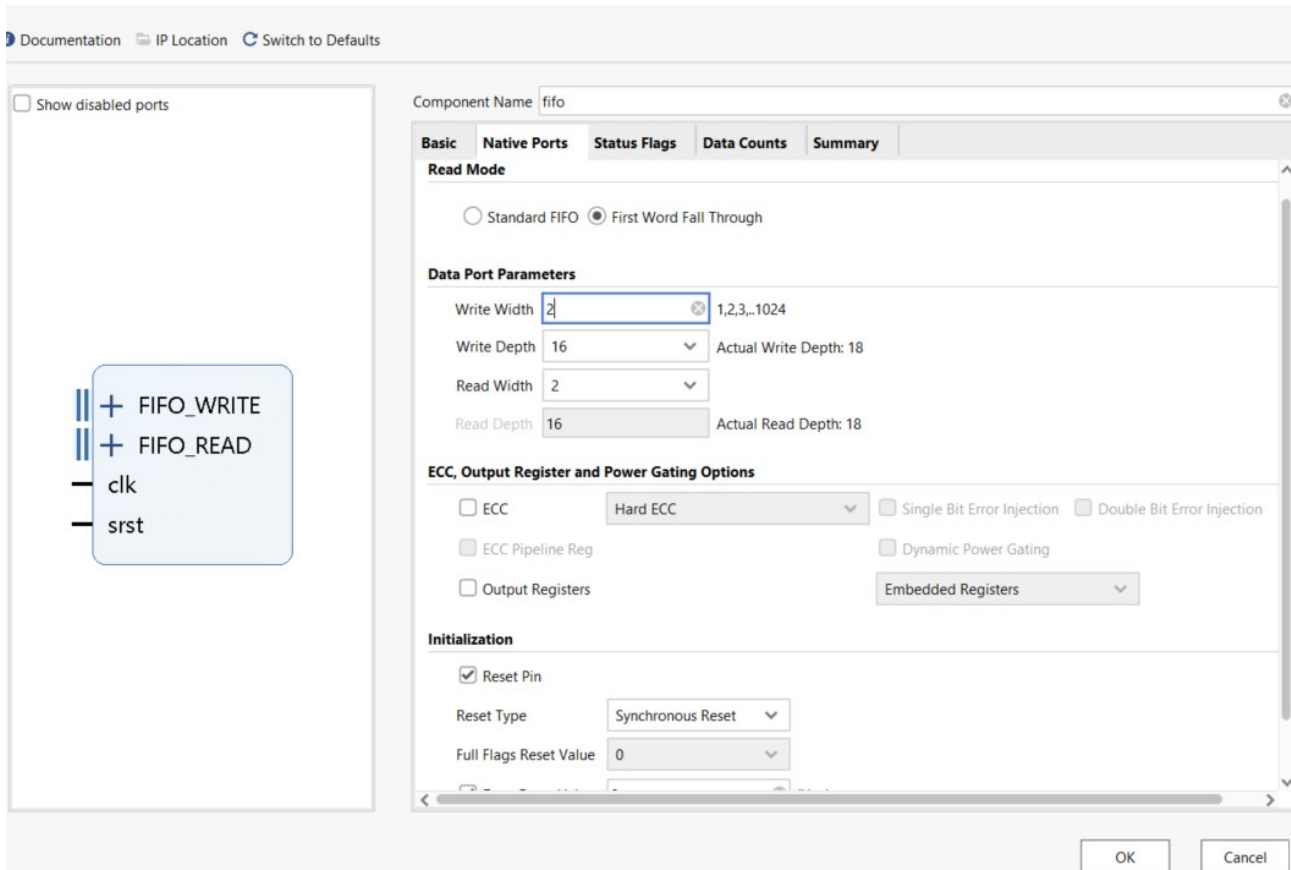
Le bloc FSM et l'entité LED_Driver dans sa globalité seront amenés à être modifié avec l'ajout d'un composant FIFO. Afin d'aller plus loin, le signal 'end_cycle' nécessite deux signaux en particulier, l'état actuel du système (**LED_ON** ou **LED_OFF**) ainsi que le flag de fin de comptage pour le clignotement des LEDs. Une logique combinatoire sera donc mis en place pour répondre à cette demande :

- L'état doit être égal à **LED_ON** (soit au niveau haut). La fin du comptage dans cet état signifie qu'un cycle ON/OFF de la LED est écoulé.
- Cela équivaut donc à cette équation :

$$\underline{end_cycle} = \underline{current_state} \text{ AND } \underline{s_end_counter}$$

3.2. Question 2

L'ajout d'une FIFO se fait via un catalogue d'IP disponible dans le logiciel Vivado tout en spécifiant les différents paramètres utiles à notre application :



La FIFO agit comme un tampon mémoire synchrone (cadencé par une horloge unique) qui stocke les commandes de couleurs transmises par l'utilisateur jusqu'à ce que le **LED_driver** soit prêt à les exécuter.

Les entrées du bloc FIFO :

- clk : Horloge système qui synchronise toutes les opérations d'écriture et de lecture
- srst : Remet la FIFO à zéro lorsqu'il passe à '**1**' (relié à not resetn)
- din (Données d'entrée - 2 bits) : Reçoit le code couleur d'abord présélectionné par le bouton_1 ("**01**" pour le vert, "**00**" pour le bleu) puis transcodé pour l'adapter aux spécifications ("**10**" pour le vert, "**11**" pour le bleu)
- wr_en (Write Enable) : Reçoit l'impulsion générée lors du front montant du 'bouton_0' pour valider l'enregistrement de 'din' dans la mémoire

- rd_en (Read Enable) : Reçoit le signal combiné 'end_cycle' and '**NOT** empty' pour valider la suppression de la couleur en cours et passer à la suivante

Les sorties du bloc FIFO :

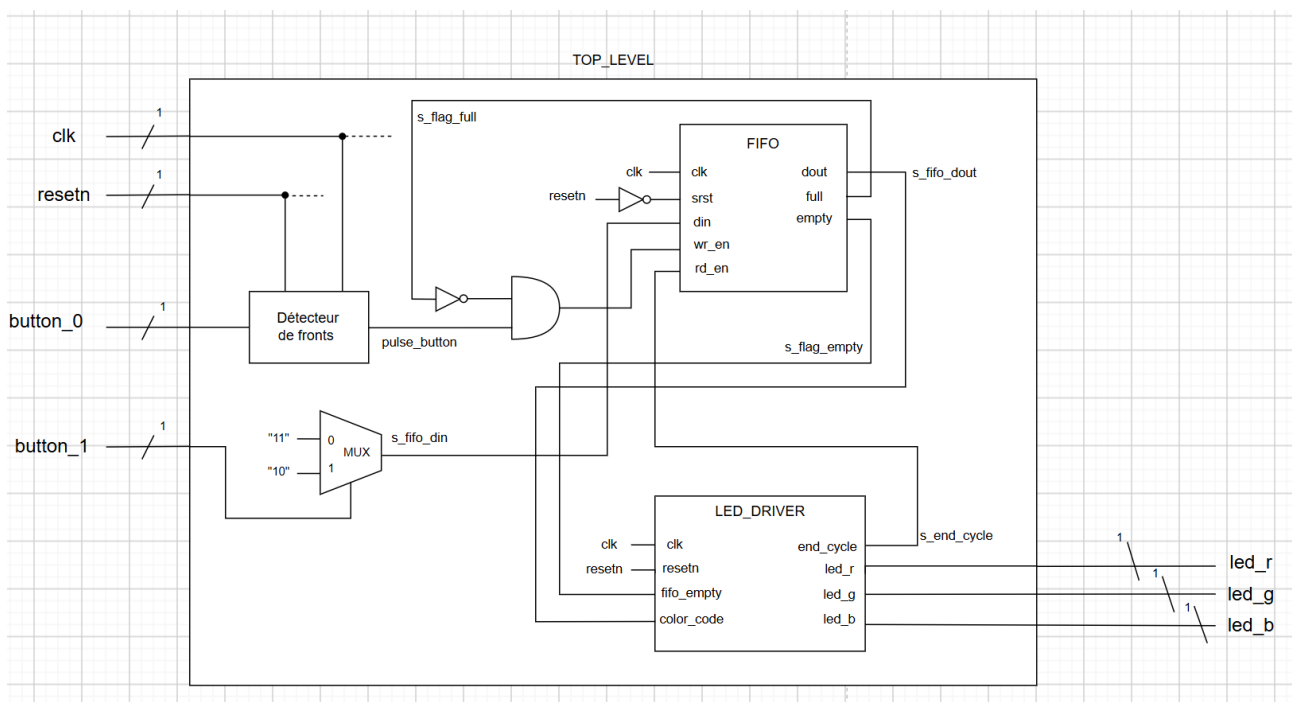
- dout (Données de sortie - 2 bits) : Fournit le code couleur à l'entrée 'color_code' du **LED_driver**. Grâce au mode **FWFT**, la couleur disponible au sommet de la **FIFO** apparaît immédiatement sur cette sortie sans attente
- full (Flag mémoire pleine) : Indique par un '**1**' logique que la mémoire a atteint sa capacité maximale (ex: 16 entrées pour notre application), interdisant tout nouvel ajout
- empty (Flag mémoire vide) : Indique par un '**1**' logique qu'aucune commande n'est en attente dans la FIFO, ce qui permet de bloquer la lecture ('rd_en')

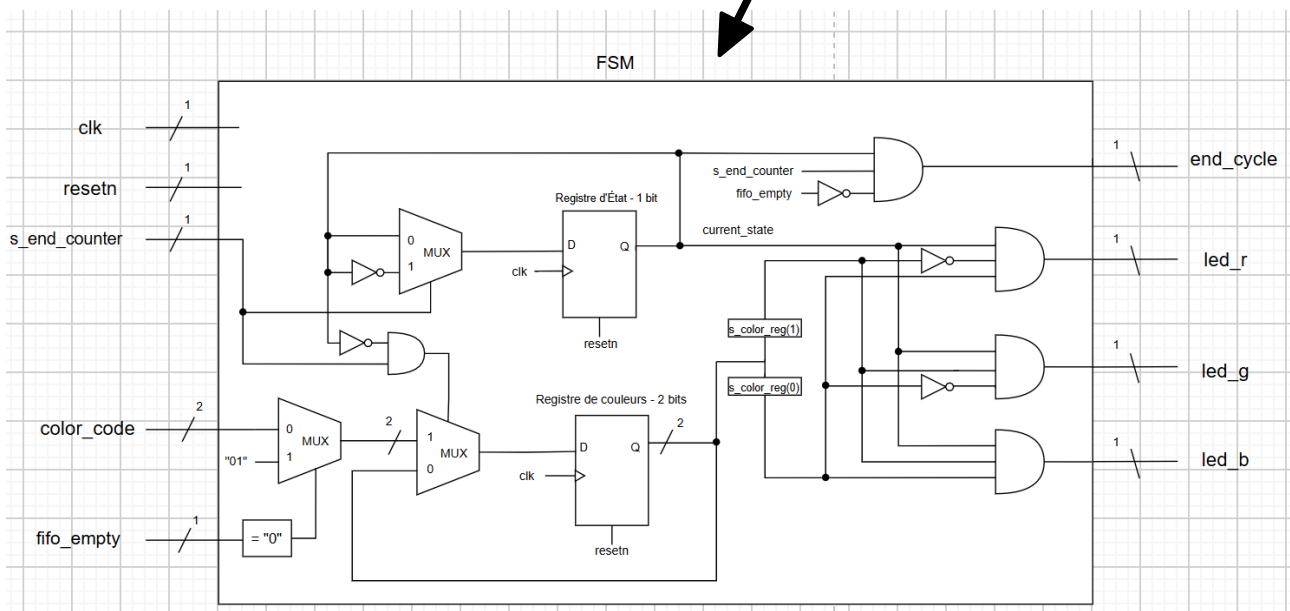
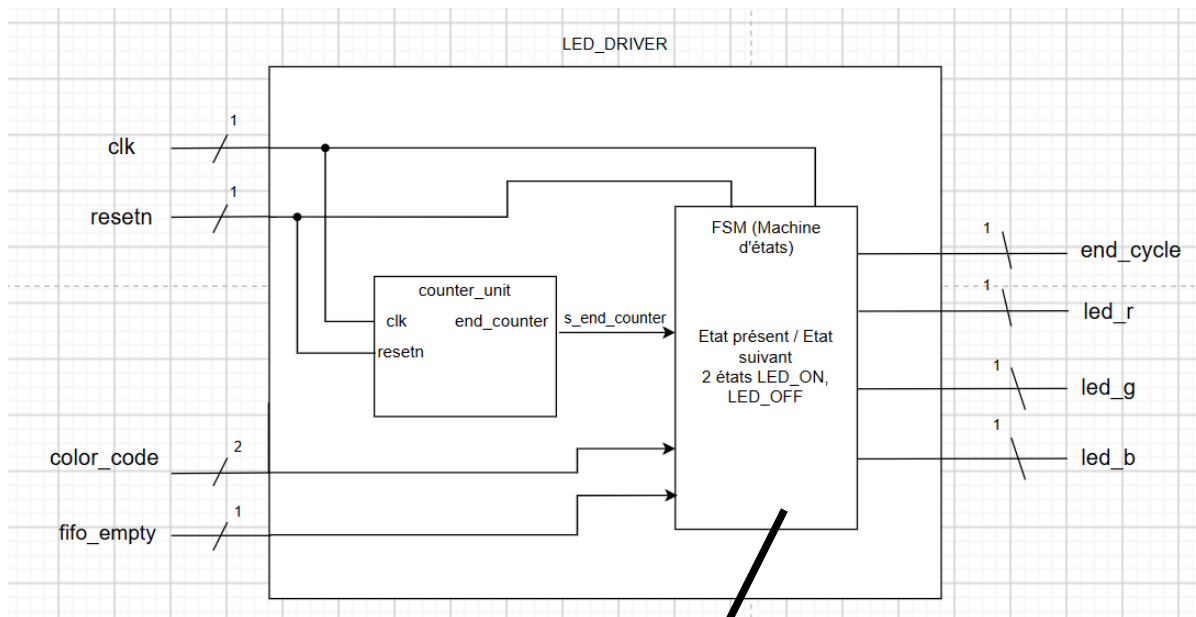
Le composant sera configuré en mode **FWFT** (First-Word Fall-Through) : dès qu'un code couleur y est écrit, il apparaît immédiatement sur la sortie 'dout'. Le LED_driver dispose ainsi en permanence de la couleur active à afficher sans cycle d'attente.

En fin de clignotement, l'impulsion 'end_cycle' (reliée à 'rd_en') dépile la donnée traitée et présente la suivante sur 'dout'. Ce fonctionnement simplifie la gestion de la lecture par rapport au mode standard.

3.3. Question 3

La structure de cette nouvelle application sera configurée autour de deux blocs principaux, la **FIFO** et le module **LED_DRIVER**. Ces deux entités seront connectées sur une couche supérieure, nommée « **TOP_LEVEL** » :





Les différentes représentations RTL ci-dessus appuient les nouvelles modifications effectuées. Concrètement, certaines fonctionnalités ont été amenés à changer de placement pour répondre aux spécifications demandées :

- **Le détecteur de fronts** : Le bouton 0 est une entrée physique nécessaire pour l'autorisation d'écriture dans la FIFO. Le bloc est ainsi placé avant l'entrée 'wr_ena'. Cependant, l'écriture dans la zone mémoire ne peut s'effectuer que si la FIFO n'est pas encore pleine → ajout d'un **flag Full** liée à la sortie 'Full' de la FIFO.
- **Le transcodeur RGB** : Le bouton 1 est une entrée physique envoyant des niveaux haut ou bas. Cette valeur est ensuite transcodée en code couleur de 2 bits (« **10** » ou « **11** »). Le signal interne reliée à la sortie du multiplexeur est reliée à l'entrée 'din' de

la FIFO afin d’être envoyée plus tard au module LED Driver par l’intermédiaire du signal de sortie ‘dout’ pour la gestion des LEDs.

Le module **LED_Driver** se retrouve donc plus allégé mais présentant tout de même quelques modifications :

- La sortie ‘end_cycle’ est utilisé tel un flag pour autoriser le dépilement et la lecture de données stockées dans la FIFO. Théoriquement, il se met au niveau haut lorsqu’un cycle OFF/ON est terminé. Néanmoins, un flag est également nécessaire afin de vérifier que la FIFO ne soit pas **vide**. Un coup de ‘rd_en’ dans une FIFO vide peut amener à une erreur d'underflow (dépileage dans le vide), pouvant planter la logique interne de l'IP FIFO.
- La valeur du registre de couleurs dépend de l’entrée color_code envoyée par la FIFO (Couleurs **verte** ou **bleue**) ou de la couleur par défaut : le **rouge**. Une LED par défaut est utilisée dans le cas où la FIFO serait vide, incluant également les situations de reset hardware. La prochaine valeur du registre est anticipée avant la toute fin d’un cycle OFF/ON afin d’obtenir cette dernière avant le début d’un cycle.

La gestion des sorties LEDs reste identique à celle présentée précédemment.

Cette nouvelle configuration sera implemtnée en 3 fichiers .vhd afin de dissocier les entités et le top level. :

TOP_LEVEL.VHD

```
-- Entité top_level de l'application : Entrées et sorties physiques + Modules FIFO & LED Driver
entity top_level is
  generic(
    TOGGLE_DELAY : integer := 999999999
  );
  Port ( clk : in std_logic;
        resetn : in std_logic;
        button_0 : in std_logic; --Update/Enable
        button_1 : in std_logic; -- Code couleur
        led_r : out std_logic;
        led_b : out std_logic;
        led_g : out std_logic);
end top_level;

architecture Behavioral of top_level is
  component led_driver is
    generic(
      TOGGLE_DELAY : integer --Horloge à 100 MHz -> 1 seconde pour chaque changement d'état
    );
    port (
      clk : in std_logic;
      resetn : in std_logic;

      fifo_empty : in std_logic; -- '0' = une donnée est disponible
      color_code : in std_logic_vector(1 downto 0); -- Le code couleur lu (ex: "10" ou "11")

      end_cycle : out std_logic; -- Signal de fin de cycle (relié au wr_en de la FIFO)
      led_r : out std_logic;
      led_b : out std_logic;
      led_g : out std_logic
    );
  end component;

  component fifo is
    PORT (
      clk : IN STD_LOGIC;
      rst : IN STD_LOGIC;
      din : IN STD_LOGIC_VECTOR(1 DOWNTO 0);
      wr_en : IN STD_LOGIC;
      rd_en : IN STD_LOGIC;
      dout : OUT STD_LOGIC_VECTOR(1 DOWNTO 0);
      full : OUT STD_LOGIC;
      empty : OUT STD_LOGIC
    );
  end component;
```

```

-----
-- Signaux internes reliant les instances
-----
signal s_end_cycle, s_wr_ena : std_logic;
signal pulse_button, s_prev_button_0 : std_logic;
signal s_fifo_dout, s_fifo_din : std_logic_vector(1 downto 0);
signal s_flag_empty, s_flag_full : std_logic;
signal fifo_reset : std_logic;

begin

    process(clk, resetn)
    begin
        if (resetn = '0') then
            s_prev_button_0 <= '0';
        elsif rising_edge(clk) then
            s_prev_button_0 <= button_0;
        end if;
    end process;

    -- Impulsion de 1 cycle au moment exact où le bouton est appuyé
    pulse_button <= button_0 and (not s_prev_button_0);

    -----
    -- Transcodage des couleurs : '0' -> "10" (Vert), '1' -> "11" (Bleu)
    -----
    s_fifo_din <= "10" when (button_1 = '1') else "11";

    -- Sécurité Overflow (Full) : Autorisation de l'écriture QUE SI appui bouton ET F
    s_wr_ena <= pulse_button and (not s_flag_full);

    fifo_reset <= not resetn;

    -----
    -- Instanciations de entités
    -----
    instance_fifo : fifo
    port map (
        clk      => clk,
        srst     => fifo_reset,
        din      => s_fifo_din,      -- Toujours en fonctionnement
        wr_en    => s_wr_ena,        -- Écriture sécurisée par le 1
        rd_en    => s_end_cycle,    -- Ordre de dépileage venant
        dout     => s_fifo_dout,
        full     => s_flag_full,    -- Flag de sécurité FIFO
        empty    => s_flag_empty   -- Flag de sécurité FIFO
    );

    instance_led_driver : led_driver
    generic map (
        TOGGLE_DELAY => TOGGLE_DELAY
    )
    port map(
        clk      => clk,
        resetn   => resetn,
        fifo_empty => s_flag_empty,
        color_code => s_fifo_dout,
        end_cycle => s_end_cycle,
        led_r     => led_r,
        led_g     => led_g,
        led_b     => led_b
    );

end Behavioral;

```

L'entité **Top_level** retranscrit le schéma RTL avec les deux instances correspondantes et reliées entre elles à l'aide de signaux internes. Les différentes logiques combinatoires comme l'autorisation d'écriture ainsi que l'anti-maintien de bouton sont également ajoutés.

LED_DRIVER.VHD (uniquement les changements effectués)

```
entity led_driver is
  generic(
    TOGGLE_DELAY : integer := 99999999 --Horloge à 100 MHz -> 1 seconde pour chaque changement d'état
  );
  port (
    clk      : in std_logic;
    resetn   : in std_logic;

    fifo_empty : in std_logic;
    color_code  : in std_logic_vector(1 downto 0); -- Le code couleur lu (ex: "10" ou "11")

    end_cycle : out std_logic; -- Signal de fin de cycle (relié au rd_en de la FIFO)
    led_r : out std_logic;
    led_b : out std_logic;
    led_g : out std_logic
  );
end led_driver;

--Logique combinatoire
-- Flag pour un cycle complet ON/OFF
s_end_cycle <= '1' when (s_end_counter = '1' and current_state = led_on and fifo_empty =
end_cycle <= s_end_cycle;

--Unité de controle synchronisant la machine d'états à l'horloge
Control_Unit : process(clk,resetn)
begin
  if(resetn = '0') then

    current_state <= led_off;
    s_color_reg <= "01"; -- LED Rouge par défaut

  elsif(rising_edge(clk)) then
    current_state <= next_state;

    if(current_state = led_on and s_end_counter = '1') then
      if(fifo_empty = '0') then
        s_color_reg <= color_code;
      else
        s_color_reg <= "01";
      end if;
    end if;
  end if;
end if;
end process Control_Unit;
```

L'application est divisée en trois fichiers sources .vhd. Quelques précisions peuvent s'ajouter pour l'implémentation VHDL :

- Le process **Outputs** se limite au décodage d'affichage (LEDs RGB). Les signaux de pilotage de la FIFO ('end_cycle'/'rd_en') sont générés à l'extérieur, en combinatoire pur, pour séparer clairement logique d'affichage et logique d'interface mémoire, et éviter d'alourdir la liste de sensibilité du process.
- 'end_cycle' doit rester combinatoire, pas enregistré : la FIFO échantillonne 'rd_en' sur front montant, donc ce signal doit déjà être stable avant ce front. En le rendant synchrone, il faudrait un cycle pour que 'end_cycle' passe à '1', puis un second pour que la FIFO en tienne compte, un cycle de latence parasite qui décalerait la capture de couleur.
- La sortie 'end_cycle' est câblée, dans **TOP_LEVEL**, sur l'entrée 'rd_en' de la FIFO : c'est l'ordre qui lui est transmis pour qu'elle avance à la donnée suivante. Sans lui, la FIFO resterait bloquée indéfiniment sur sa première valeur. Au même front d'horloge où ce signal passe à '1', le module **LED_Driver** capture la valeur présente sur 'dout', tandis que la FIFO efface cette même donnée et prépare la suivante. Cette simultanéité garantit l'alignement de dépilement et capture décrit plus haut.

3.4. Question 4

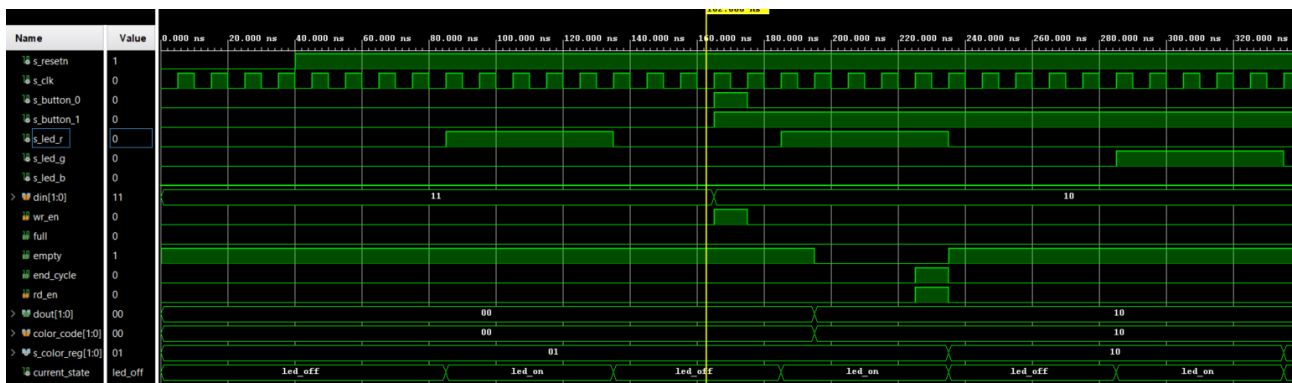
Le testbench ***tb_top_level*** valide le système complet en quatre étapes :

- Reset et comportement par défaut : vérification que la LED Rouge s'allume automatiquement après reset, la FIFO étant vide.
- Sélection d'une couleur par appui bref : un appui d'un cycle déclenche l'écriture en FIFO et l'affichage de la couleur correspondante.
- Test de maintien prolongé : le bouton reste actif plusieurs cycles pour vérifier qu'aucune écriture répétée ne se produit tant qu'il n'y a pas de nouveau front montant.
- Remplissage successif de la FIFO : plusieurs couleurs sont écrites rapidement à la suite, pour valider que le système les restitue dans leur ordre exact d'écriture.

Une fonctionnalité a été ajoutée pour simplifier le code : une **procédure** `press_button`, qui factorise la séquence d'appui bref (`button_0` à '1' pendant un cycle puis relâchement) en un seul appel réutilisable, évitant de dupliquer ces lignes à chaque test.

Le code VHDL du test bench est visualisable dans le fichier « `tb_top_level.vhd` »

Phase 1 -- Comportement par défaut et allumage LED Verte



Après le relâchement de `resetn`, le système démarre en **LED_OFF** avec la FIFO vide (`'empty' = '1'`, `'full' = '0'`). Le signal `din` affiche déjà "11" en continu : c'est normal, `'din'` est purement combinatoire sur `'button_1'` (valeur par défaut '0'), mais cette valeur n'est pas encore écrite puisque `'wr_en' = '0'`, elle est simplement « en attente » sur le bus d'entrée de la FIFO, sans conséquence tant qu'aucune impulsion n'arrive.

La FSM bascule en **LED_ON** avec `'s_color_reg' = "01"` (chargé par défaut au reset, puisque la FIFO était vide). `'led_r'` passe à '1', confirmant l'affichage du Rouge par défaut.

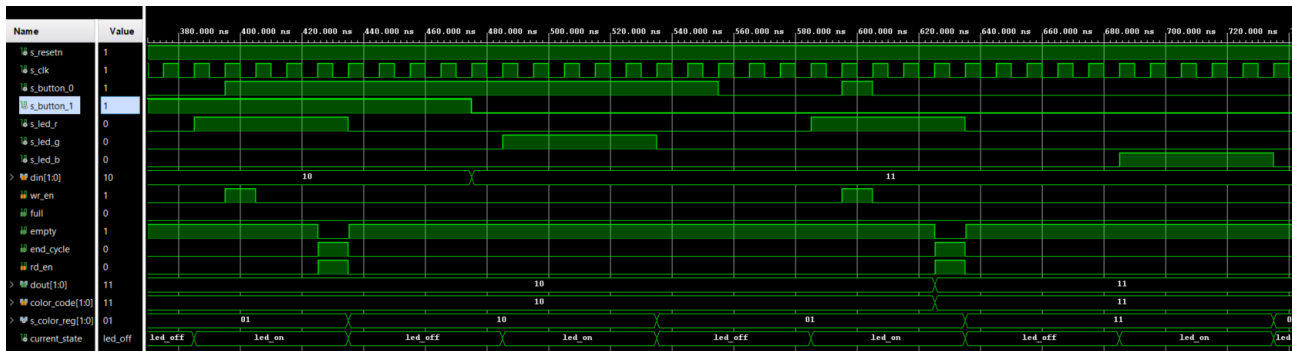
`'s_button_0'` pulse pendant exactement 1 cycle d'horloge, avec `'s_button_1'` à 1 positionné simultanément (sélection Vert). Cette impulsion se propage directement sur `'wr_en'`, qui pulse également 1 cycle. C'est la preuve visuelle que `'pulse_button'` fonctionne comme prévu : une seule impulsion nette, sans répétition.

Aussitôt après cette écriture, 'empty' retombe à '0' : la FIFO contient désormais une donnée ("10", Vert) en attente de lecture.

À la fin de la phase **LED_ON** en cours, 'end_cycle' (relié à 'rd_en' de la FIFO) pulse un cycle : c'est le moment précis où le module **LED_Driver** capture la couleur ('color_code' = 'dout' = "10") et demande simultanément le dépilage de cette même donnée. Ces deux signaux passent effectivement de "00" à "10" à cet instant, confirmant que la FIFO restitue bien la donnée écrite.

's_color_reg' passe de "01" à "10" au cycle suivant (registre synchrone, un cycle de latence normal après la capture). 'empty' remonte à '1' juste après ce dépilage (FIFO de nouveau vide, une seule donnée ayant été traitée). À la prochaine transition, 'led_g' s'allume enfin : la LED Verte est correctement affichée, validant la phase 1 du testbench de bout en bout : bouton → écriture FIFO → capture synchronisée → registre couleur → sortie physique.

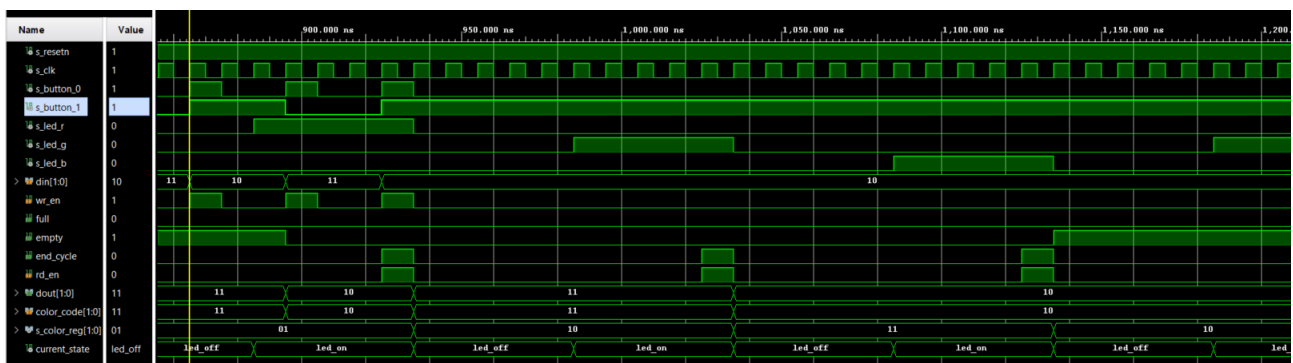
Phase 2 – Test anti-maintien et allumage LED Bleue (*Button 0 = '1' et Button 1 = '0'*)



Malgré l'appui prolongé sur 's_button_0' (maintenu à '1' avec 's_button_1' = '1' pour sélectionner le Bleu "11"), 'wr_en' ne génère strictement qu'un seul pulse d'un cycle. Cela confirme que le module 'pulse_button' bloque l'écriture continue en cas de maintien.

Sous le même principe que la phase précédente, la donnée "11" présente dans la FIFO est dépillée sur l'impulsion 'end_cycle / rd_en' après une nouvelle impulsion brève sur 'button_0'. Le registre 's_color_reg' bascule à "11", entraînant l'activation effective de la sortie 's_led_b' lors du cycle **led_on** suivant.

Phase 3 -- Stockage successif de plusieurs valeurs en entrée (Vert → Bleu → Vert)



Afin de tester la capacité de zone mémoire de la FIFO, plusieurs valeurs sont stockées à la suite via un enchaînement d'impulsions brèves sur 'button_0', avec quelques impulsions sur 'button_1' pour alterner les couleurs (Bleu, Vert, Bleu).

On observe trois pulses de 'wr_en' rapprochés, chacun empilant une nouvelle couleur dans la FIFO avant que la précédente n'ait été consommée, 'empty' ne remonte à '1' qu'après le tout dernier dépilage, confirmant que les trois données coexistent bien en mémoire entre-temps.

Le module **LED_Driver** égrène ensuite ces couleurs dans leur ordre exact d'écriture, une par une à chaque transition de l'état **LED_OFF** à l'état **LED_ON**, via la même chaîne de capture déjà validée précédemment (end_cycle/rd_en → dout/color_code → s_color_reg). Cette phase confirme que la FIFO conserve correctement l'ordre FIFO (premier écrit, premier lu) même en cas d'écritures rapprochées, sans perte ni permutation de donnée.

3.5. Question 5

Estimation théorique des ressources

Avant la synthèse, une estimation manuelle des ressources a été réalisée en décomposant le design primitive par primitive :

- Compteur (counter_unit) : registre de comptage sur 28 bits, soit environ 28 bascules, accompagnées de la logique d'incrément et de comparaison à la valeur maximale (Environ 7 blocs CARRY4 et 27-30 LUT)
- FSM (led_driver, 2 états) : 1 bascule pour l'état courant, 2 bascules pour le registre 's_color_reg', quelques LUT pour la logique de transition et de sortie combinatoire
- Détection de front : 1 bascule ('s_prev_button_0') et quelques LUT pour 'pulse_button', le mux de transcoding couleur, et la sécurité anti-overflow ('s_wr_ena')
- FIFO (IP Xilinx, profondeur 16×2 bits, FWFT) : pointeurs de lecture/écriture (Environ 10 bascules), logique de comparaison des flags full/empty, registre de sortie FWFT, et mémoire de stockage estimée en LUTRAM (Environ 2 LUT) pour une aussi petite profondeur
- Primitives d'E/S : 4 IBUF (clk, resetn, button_0, button_1), 3 OBUF (led_r, led_g, led_b), 1 BUFG pour la distribution d'horloge

Total estimé : Environ 41 bascules, 53 LUT (dont 2 LUTRAM), 4 IBUF, 3 OBUF, 1 BUFG

Rapport post-synthèse

7. Primitives

Ref Name	Used	Functional Category
LUT4	35	LUT
FDCE	31	Flop & Latch
CARRY4	7	CarryLogic
IBUF	4	IO
OBUF	3	IO
LUT6	3	LUT
LUT5	3	LUT
LUT3	3	LUT
LUT1	3	LUT
LUT2	1	LUT
FDPE	1	Flop & Latch
BUFG	1	Clock

7. Primitives

Ref Name	Used	Functional Category
FDRE	21	Flop & Latch
LUT4	9	LUT
FDSE	9	Flop & Latch
LUT5	6	LUT
LUT3	5	LUT
LUT1	5	LUT
LUT2	3	LUT
RAMB18E1	1	Block Memory
LUT6	1	LUT

**Rapport de synthèse - TOP_LEVEL (SANS
FIFO)**

Rapport de synthèse - FIFO

Le rapport post-synthèse confirme un total de 62 bascules et 77 LUT (32 FF / 48 LUT pour **LED_DRIVER**, 30 FF / 29 LUT pour la FIFO), contre une estimation initiale de 41 FF et 53 LUT. Les ressources d'entrée/sortie (4 IBUF, 3 OBUF, 1 BUFG) correspondent exactement à la prévision.

L'écart observé sur les FF et les LUT (facteur d'environ 1,5x) reste cohérent avec l'estimation et s'explique par deux causes identifiées : la logique de comparaison et de propagation de retenue du compteur (CARRY4, LUT associées) légèrement plus consommatrice que prévu à ce gabarit, et le choix de l'outil d'implémenter la mémoire de la FIFO en bloc RAMB18E1 plutôt qu'en LUTRAM, une décision propre à la configuration par défaut de l'IP Xilinx FIFO Generator et difficile à anticiper sans en inspecter les paramètres. L'estimation théorique est donc jugée **globalement conforme** au résultat réel, les écarts constatés étant expliqués plutôt que résultant d'une erreur de méthode.

3.6. Question 6

Le rapport d'implémentation final montre un respect parfait des contraintes de timing :

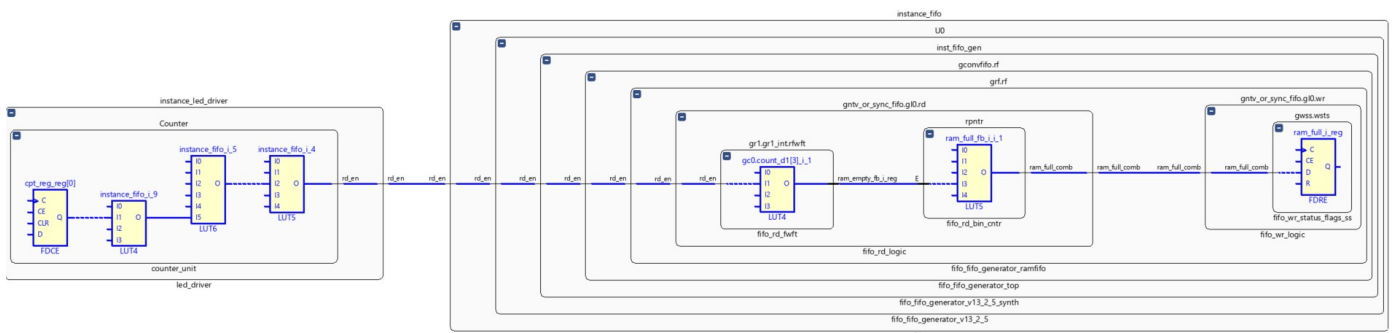
Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 3,326 ns	Worst Hold Slack (WHS): 0,133 ns	Worst Pulse Width Slack (WPWS): 4,500 ns
Total Negative Slack (TNS): 0,000 ns	Total Hold Slack (THS): 0,000 ns	Total Pulse Width Negative Slack (TPWS): 0,000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 93	Total Number of Endpoints: 93	Total Number of Endpoints: 65

All user specified timing constraints are met.

- Worst Negative Slack (WNS) : Positif (pas de violation de Setup)
- Worst Hold Slack (WHS) : Positif (pas de violation de Hold)

Name	Slack	Levels	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock	Destination Clock	Exception	Clock Uncertainty
Path 1	3.326	5	31	instance_led_reg_reg[0]/C	instance_fifo_full_j_reg/D	6.257	1.469	4.788	10.000	sys_clk_pin	sys_clk_pin		0.035
Path 2	3.667	5	31	instance_led_reg_reg[0]/C	instance_fifo_ty_fb_j_reg/D	6.130	1.443	4.687	10.000	sys_clk_pin	sys_clk_pin		0.035
Path 3	3.835	5	31	instance_led_reg_reg[0]/C	instance_fifopty_j_reg/D	5.947	1.443	4.504	10.000	sys_clk_pin	sys_clk_pin		0.035
Path 4	3.971	5	31	instance_led_reg_reg[0]/C	instance_fifo_l_fb_j_reg/D	5.923	1.469	4.454	10.000	sys_clk_pin	sys_clk_pin		0.035
Path 5	3.972	4	31	instance_led_reg_reg[0]/C	instance_fifo_d1_reg[0]/CE	5.448	1.116	4.332	10.000	sys_clk_pin	sys_clk_pin		0.035
Path 6	3.972	4	31	instance_led_reg_reg[0]/C	instance_fifo_d1_reg[1]/CE	5.448	1.116	4.332	10.000	sys_clk_pin	sys_clk_pin		0.035



L'analyse temporelle réalisée par Vivado montre que le Path 1 constitue le chemin critique de notre design, présentant à la fois le Slack (Worst Negative Slack) le plus faible (**3,326 ns**) et le délai de propagation total le plus élevé (**6,257 ns**, se décomposant en **1,469 ns** de délai logique et 4,788 ns de délai d'interconnexion/routage).

Néanmoins, avec un Slack largement positif de **+3,326 ns** sur une période d'horloge requise de 10,0 ns (horloge système sys_clk_pin à 100 MHz), l'ensemble de nos contraintes de temps d'établissement sont parfaitement respectées, sans aucun défaut temporel.

Le chemin critique extrait (visible sur le schéma technologique ci-dessus) franchit les limites de modules : il s'étend de la bascule de départ cpt_reg_reg[0] (au sein du composant **counter_unit** du **LED_Driver**) jusqu'à la bascule d'arrivée ram_full_j_reg (dans la logique de gestion des flags de la FIFO).

Physiquement, ce chemin correspond à la propagation du signal de contrôle depuis le bit de poids faible du compteur dans le **LED_driver** jusqu'à la mise à jour des drapeaux d'état de la **FIFO**. Le signal traverse 5 niveaux de logique combinatoire (composés exclusivement de portes LUT4, LUT5 et LUT6) avant d'atteindre l'entrée de donnée du registre de destination dans la **FIFO**.

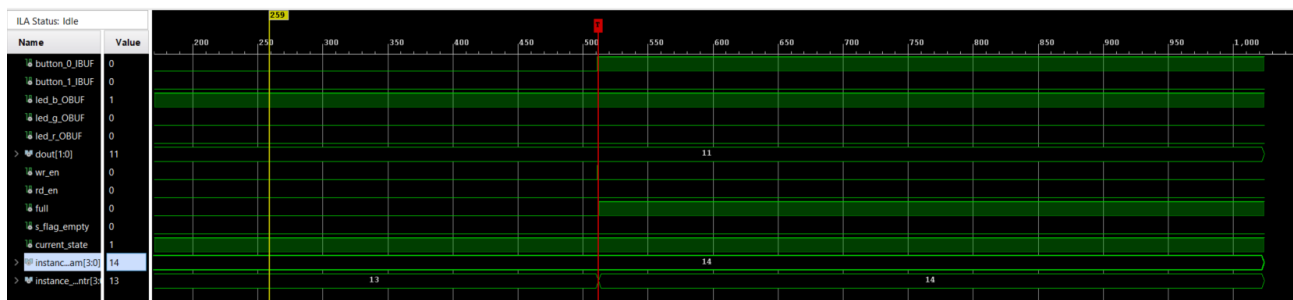
L'analyse détaillée de ce chemin critique montre que le facteur limitant n'est pas la complexité logique (le délai logique ne représentant que **1,469 ns**, soit 24 % du temps total), mais le **délai de routage (4,788 ns, soit 76 % du temps total)**.

3.7 Question 7

Une démonstration vidéo est également disponible pour montrer le bon fonctionnement de cette partie de ce TP. (disponible dans le fichier README du dossier TP4 qui dirige vers un lien Google drive).

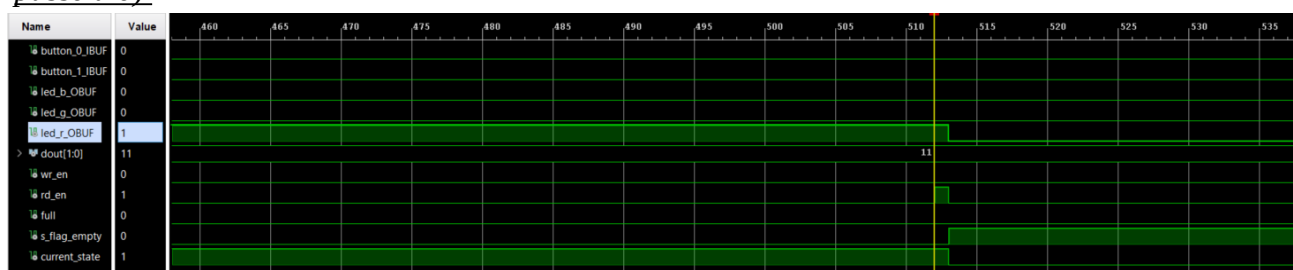
La plupart des résultats étant visible sur carte ou par simulation, quelques cas de debug seulement ont été observés :

Test du flag full – Saturation attendue du FIFO



La capacité réellement observée par sonde ILA est de 14 écritures avant assertion de 'full', contre une profondeur configurée de 16. Cet écart s'explique par le mode **FWFT** (un emplacement réservé au registre de sortie) combiné à la marge de sécurité du flag 'full', qui anticipe la saturation pour compenser la latence de calcul interne du pointeur.

Test du rd_ena full – Prise en compte de la prochaine valeur pour le cycle OFF/ON (empty passe à 0)



Limitations :

La détection de front montant (*'pulse button'*) résout le cas demandé par l'énoncé : un bouton maintenu enfoncé ne génère qu'une seule écriture en **FIFO**, quelle que soit la durée de l'appui. Une limitation non traitée dans ce TP concerne le rebond mécanique (ou bounce) des boutons physiques : lors de la fermeture/ouverture du contact, plusieurs fronts montants parasites peuvent être générés en quelques millisecondes, ce qui pourrait entraîner l'empilement de plusieurs couleurs pour un seul appui perçu par l'utilisateur. Ce phénomène nécessiterait un module anti-rebond dédié (temporisation de quelques ms avant validation du front), qui pourrait également être implémenté.